

京东零售前端开发工程师1-3面

一面面试题（技术面）

- 自我介绍一下，然后重点介绍一下你最近做的或者你认为最有技术挑战的项目。
- 看你的项目主要用 React，说一下 React 的 Class Component 和 Function Component 有什么区别。
- React Hooks 是什么？它解决了什么问题？你常用的 Hooks 有哪些？
- `useEffect` 的依赖项数组是做什么用的？如果是空数组、没有依赖项数组或者有依赖项，分别代表什么？
- `useCallback` 和 `useMemo` 的区别和使用场景是什么？
- React 的 state 和 props 有什么区别？它们是同步更新还是异步更新？
- React 的合成事件是什么？和原生事件有什么区别？
- 讲一下 React 的 key 是做什么用的？为什么不推荐用 index 作为 key？
- 说说你对 Redux 的理解，它的核心思想和基本原则是什么？中间件的机制是怎样的？
- 除了 Redux，还用过别的状态管理方案吗？比如 MobX, Zustand，它们有什么优缺点？
- JavaScript 的事件循环机制（Event Loop）是什么？讲一下宏任务和微任务。
- `this` 的指向问题，箭头函数和普通函数的 `this` 有什么不同？如何改变 `this` 的指向？
- 讲一下从输入 URL 到页面加载完成，整个过程发生了什么？
- HTTP 缓存有哪几种？分别是什么机制？
- TCP 的三次握手和四次挥手过程能说一下吗？
- CSS 的 Flex 布局 and Grid 布局你用过吗？分别适合什么场景？
- Webpack 和 Vite 的区别是什么？Vite 为什么快？
- 你在项目中做过哪些 Webpack 的配置和优化？
- 算法题：实现一个 `debounce` (防抖) 函数。
- 你有什么想问我的？

二面面试题（技术面）

- 挑一个你最复杂的项目，画一下它的整体架构图，讲一下你在里面负责什么，解决了什么核心问题？
- 你在做这个项目时，技术选型是怎么考虑的？为什么选择 React 和你们用的那个状态管理库？
- React 的 Fiber 架构是什么？它解决了什么问题？
- React 的 Diff 算法是怎么工作的？和 Vue 的 Diff 算法有什么区别？
- 如何对一个 React 应用进行性能优化？从你的项目实践出发，具体谈谈。
- 如果一个列表页，有大量数据需要渲染，如何防止页面卡顿？
- 你对前端工程化是怎么理解的？你在团队里是如何实践的？
- 你们的代码质量是如何保障的？比如 Code Review, ESLint, Unit Test 等。
- 讲一下微前端，了解吗？它的实现方案有哪些？
- SSR（服务端渲染）和 CSR（客户端渲染）有什么区别？你在项目中用过吗？
- Web 安全问题，除了 XSS 和 CSRF，还了解其他的攻击方式吗？比如点击劫持。
- HTTPS 的加密过程是怎样的？为什么需要数字证书？
- Node.js 在前端开发中扮演什么角色？你用它做过什么？
- 场景题：现在要实现一个大型电商网站的商品详情页，你会如何设计这个页面？需要考虑哪些技术点，比如性能、可用性、SEO 等。
- 算法题：给定一个无序的整数数组，找到其中最长连续序列的长度。

三面面试题（总监/经理面）

- 简单聊聊你过去几年的工作经历和成长。
- 在你之前的项目里，你觉得最有挑战的事情是什么？你是如何推动解决的？
- 你如何看待技术深度和技术广度的关系？
- 如果让你来带一个项目，你会从哪些方面来保证项目的顺利进行？
- 当你的技术方案和团队里其他人的想法不一致时，你会怎么处理？
- 你对前端行业的未来发展趋势有什么看法？
- 你平时是如何学习新技术的？最近在关注什么新技术？
- 你认为自己的核心竞争力是什么？
- 聊聊你的职业规划，未来 3-5 年有什么打算？
- 你在团队协作和沟通上，有什么自己总结的方法论吗？
- 你觉得我们这个业务，前端可以做哪些事情来创造更大的价值？

一面参考答案：

2.React 的 Class Component 和 Function Component 有什么区别？

Class Component 和 Function Component 是 React 中创建组件的两种主要方式，它们在语法、状态管理、生命周期和性能等方面存在显著差异。

特性	Class Component (类组件)	Function Component (函数组件)
语法	基于 ES6 的 class 语法，需要继承 React.Component，并实现 render() 方法。	就是一个普通的 JavaScript 函数，接收 props 作为参数，返回 JSX。语法更简洁。
状态管理 (State)	通过 this.state 来定义和访问 state，使用 this.setState() 方法来更新。	通过 useState Hook 来管理 state。
生命周期	拥有一套完整的生命周期方法，如 componentDidMount, componentDidUpdate, componentWillUnmount 等。	没有传统的生命周期方法，通过 useEffect Hook 来模拟和实现类似的功能（如组件挂载、更新、卸载）。
this 指向	内部 this 的指向比较复杂，常常需要在构造函数中 bind 事件处理函数，或者使用箭头函数来避免 this 指向问题。	没有 this 的困扰，因为函数组件内部没有自己的 this。
逻辑复用	主要通过高阶组件 (HOC) 和 Render Props 来复用逻辑，但这两种模式都可能导致组件层级过深 (Wrapper Hell) 。	通过自定义 Hooks (Custom Hooks) 来复用状态逻辑，代码更扁平、更清晰、更易于维护。
未来趋势	仍然被支持，但在 React 官方推荐中，函数组件和 Hooks 是未来的主流方向。	React 官方主推的组件编写方式，拥有更好的性能潜力和优化空间。

总的来说，Function Component + Hooks 的组合使得组件逻辑更清晰、更易于复用，并且避免了 Class Component 中 `this` 的复杂性，是目前社区和官方推荐的最佳实践。

React Hooks 是什么？它解决了什么问题？你常用的 Hooks 有哪些？

React Hooks 是 React 16.8 版本引入的一项新特性。它允许你在不编写 class 的情况下使用 state 以及其他的 React 特性。本质上，Hooks 就是一些特殊的函数，可以让你“钩入” React 的 state 及生命周期等特性。

它主要解决了以下几个问题：

1. **在组件之间复用状态逻辑很难：** 过去我们主要使用 Render Props 和高阶组件（HOC）来复用逻辑，但这两种模式都会增加额外的组件层级，形成“嵌套地狱”，使代码难以理解。自定义 Hooks 可以让你抽离和复用状态逻辑，而无需改变组件结构。
2. **复杂组件变得难以理解：** 在 Class Component 中，相关的逻辑（例如，在 `componentDidMount` 中订阅事件，在 `componentWillUnmount` 中取消订阅）被强制分散在不同的生命周期函数里，而不相关的逻辑（例如，在 `componentDidMount` 中同时获取数据和添加事件监听）却被组合在一起。`useEffect` Hook 可以让你根据逻辑相关性来组织代码，而不是根据生命周期方法。
3. **难以理解的 `this`：** Class Component 中的 `this` 关键字是 JavaScript 的一个难点，它在事件处理函数中的指向问题经常导致 bug，需要开发者手动绑定或使用特殊语法。Function Component 则没有 `this` 的困扰。

我常用的 Hooks 有：

- `useState`：用于在函数组件中添加和管理 state。
- `useEffect`：用于处理副作用，如数据获取、订阅、手动修改 DOM 等。它可以看作是 `componentDidMount`，`componentDidUpdate` 和 `componentWillUnmount` 这三个生命周期函数的组合。
- `useContext`：用于在组件树中进行跨层级的状态共享，避免 props 的层层传递。
- `useRef`：用于获取 DOM 节点的引用，或者存储一个在多次渲染之间保持不变的可变值（这个值的改变不会触发组件重新渲染）。
- `useMemo`：用于缓存计算结果。它会记住一个昂贵计算的返回值，只有当依赖项发生变化时才重新计算。
- `useCallback`：用于缓存函数本身。它会返回一个 memoized 版本的函数，这个函数只有在依赖项改变时才会更新。常用于将函数传递给子组件以避免不必要的重新渲染。
- `useReducer`：`useState` 的替代方案，更适合用于管理包含多个子值的复杂 state 逻辑，或者下一个 state 依赖于前一个 state 的场景。

`useEffect` 的依赖项数组是做什么用的？如果是空数组、没有依赖项数组或者有依赖项，分别代表什么？

`useEffect` 的依赖项数组（Dependency Array）是它的第二个参数，它告诉 React **只有当数组中的值发生变化时，才需要重新执行** `useEffect` 内部的副作用函数。这是一种性能优化的关键手段，可以避免在每次组件渲染时都执行不必要的副作用。

三种情况的区别如下：

1. 有依赖项数组 (`[dep1, dep2, ...]`):

- **行为：**副作用函数会在组件**首次渲染**时执行一次。之后，**只有当依赖项数组中的任何一个值与上一次渲染时相比发生了变化**，副作用函数才会再次执行。
- **用途：**这是最常见的用法。用于那些依赖于特定 `props` 或 `state` 的副作用。例如，当一个组件的 ID prop 变化时，重新去获取该 ID 对应的数据。

代码块

```
1  useEffect(() => {
2    // 当 userId 变化时，重新获取用户数据
3    fetchUserData(userId);
4  }, [userId]);
```

1. 空数组 (`[]`):

- **行为：**副作用函数**只会在组件首次挂载（mount）到 DOM 后执行一次**。它不会在后续的任何重新渲染中再次执行。如果 `useEffect` 返回了一个清理函数，这个清理函数会在组件卸载（unmount）时执行。
- **用途：**非常适合执行只需要运行一次的副作用。例如，设置事件监听器、启动定时器、或者进行初始的数据获取。

代码块

```
1  useEffect(() => {
2    // 只在组件挂身时添加事件监听window.addEventListener('scroll', handleScroll);
3    // 在组件卸载时移除监听，防止内存泄漏return () => {
4      window.removeEventListener('scroll', handleScroll);
5    };
6  }, []); // 空数组表示不依赖任何 props 或 state
```

1. 没有依赖项数组 (`undefined`):

- **行为：**副作用函数**在每次组件渲染（包括首次渲染和后续的所有更新）之后都会执行**。

- **用途：**这种用法需要非常小心，因为它很容易导致性能问题或无限循环。通常应该避免这种写法，除非你明确知道每次渲染都需要重新运行这个副作用。

代码块

```
1  useEffect(() => {  
2    // 每次组件渲染后都会执行，可能会造成性能问题 console.log('Component re-rendered');  
3  });
```

useCallback 和 useMemo 的区别和使用场景是什么？

`useCallback` 和 `useMemo` 都是用于性能优化的 Hooks，它们的核心思想都是“记忆化”（memoization），即缓存某些东西以避免在每次渲染时都重新创建。

核心区别：

- `useMemo` **缓存的是计算结果（值）**。它会执行一个函数并记住它的返回值。只有当依赖项数组中的值发生变化时，它才会重新执行该函数并返回新的值。
- `useCallback` **缓存的是函数本身（引用）**。它会返回一个函数的 memoized 版本。这个被缓存的函数只有在依赖项数组中的值发生变化时才会更新。

可以这样理解：`useCallback(fn, deps)` 等价于 `useMemo(() => fn, deps)`。

使用场景：

- `useMemo` 的使用场景：
 - **缓存昂贵的计算：**当你的组件中有一些计算量很大的操作（比如对一个大数组进行过滤、排序、映射），并且这个操作的依赖项不经常变化时，应该使用 `useMemo` 来缓存计算结果，避免在每次渲染时都重复执行这些昂贵的计算。

代码块

```
1  const visibleTodos = useMemo(() => {  
2    // 这是一个昂贵的计算 return filterTodos(todos, tab);  
3  }, [todos, tab]);
```

- `useCallback` 的使用场景：

- **将函数传递给被优化的子组件：**这是最主要的使用场景。当一个父组件将一个函数作为 prop 传递给一个被 `React.memo` 包裹的子组件时，如果这个函数是在父组件中直接定义的，那么每次父组件重新渲染时，这个函数的引用都会改变，从而导致子组件即使接收到的其他 props 没有变化，也会不必要地重新渲染。使用 `useCallback` 可以保证只要依赖项不变，传递给子组件的函数引用就不会变，从而避免了子组件的无效渲染。

代码块

```
1  const handleAddItem = useCallback(() => {
2    setItems([...items, 'new item']);
3  }, [items]); // 只有当 items 数组变化时，函数才会被重新创建
return <TodoList onAddItem={handleAddItem} />; // 传递给子组件
```

React 的 state 和 props 有什么区别？它们是同步更新还是异步更新？

区别：

特性	Props (Properties)	State
数据来源	由父组件传递下来，是外部传入的数据。	组件内部自身管理的数据。
可变性	不可变的 (Immutable) 。组件内部不能直接修改 props。这是一个单向数据流的核心原则。	可变的 (Mutable) 。组件可以通过 <code>setState</code> 或 <code>useState</code> 的更新函数来修改 state。
作用	用于组件之间的通信，让组件可以从外部接收配置和数据。	用于管理组件的内部状态，驱动组件 UI 的变化。
所有权	属于父组件。	属于组件自身。

简单来说：`props` 是从外面传进来的，`state` 是在组件里面自己生成的。

更新机制：同步还是异步？

React 的 `setState`（或 `useState` 的更新函数）在大部分情况下表现为**异步更新**。

- 为什么是异步的？

- React 为了性能优化，会将多个 `setState` 调用合并（batching）成一个单独的更新。例如，在一个事件处理函数中连续调用多次 `setState`，React 不会每调用一次就重新渲染一次组件，而是会将这些更新收集起来，在事件处理函数执行完毕后，再一次性地进行 DOM diff 和重新渲染。这可以避免不必要的渲染，提升性能。

代码块

```
1  function handleClick() {
2    // 这三个 setState 调用会被合并成一次更新
3    setCount(c => c + 1);
4    setFlag(f => !f);
5    setName('new name');
6    // 在这里立即访问 state，获取到的仍然是旧的值 console.log(count); // 打印的是更新前的值
7  }
```

- 什么时候是同步的？
- 在 React 的事件处理函数（如 `onClick`, `onChange`）和生命周期函数（或 `useEffect`）之外的异步代码中，比如 `setTimeout`, `setInterval`，或者原生 DOM 事件监听器中调用 `setState`，更新通常是同步的。这是因为在这些上下文中，React 无法控制执行时机，也就无法进行批量更新。
- 不过，从 React 18 开始，通过自动批处理（Automatic Batching），即使是在 `setTimeout` 或 `Promise` 回调中，更新也会被自动批量处理，表现为异步。因此，最佳实践是：始终认为 `setState` 是异步的，并且不要依赖它来立即获取更新后的 `state`。如果需要基于更新后的 `state` 执行操作，应该使用 `useEffect` 或者 `setState` 的回调函数形式。

React 的合成事件是什么？和原生事件有什么区别？

React 合成事件（SyntheticEvent） 是 React 自己实现的一套事件系统。它是对浏览器原生事件的跨浏览器包装器。当你为 JSX 元素添加事件处理器（如 `onClick`）时，你接收到的 `event` 对象就是一个 `SyntheticEvent` 实例。

和原生事件的区别：

1. 跨浏览器兼容性：

- **合成事件：**React 抹平了不同浏览器之间原生事件的差异。例如，不同浏览器对某些事件的属性命名可能不同，React 的 `SyntheticEvent` 提供了统一的 API，让你无需关心这些兼容性问题。
- **原生事件：**开发者需要自己处理浏览器的兼容性问题。

2. 事件委托机制：

- **合成事件**：React 并不会将事件处理器直接绑定到真实的 DOM 元素上。相反，它在 `document` 根节点上（React 17 之后是应用的根节点）使用一个统一的事件监听器。当事件触发时，它会利用事件冒泡机制，在根节点捕获事件，然后根据 `event.target` 等信息，找到真正触发事件的 React 组件，并执行对应的事件处理器。这种机制被称为**事件委托**。
- **原生事件**：通常是直接将事件监听器绑定在具体的 DOM 元素上。

3. 性能：

- **合成事件**：由于采用了事件委托和事件对象池（Event Pooling）机制（React 17 之后移除了对象池），减少了内存占用和事件注册的数量，从而在一定程度上提升了性能。
- **原生事件**：如果大量元素都绑定事件，可能会占用更多内存，并可能影响初始渲染性能。

4. 事件对象：

- **合成事件**：传递给事件处理函数的是 `SyntheticEvent` 对象。如果你想访问原生的事件对象，可以通过 `event.nativeEvent` 属性来获取。
- **原生事件**：传递的是浏览器原生的 `Event` 对象。

总结来说，React 合成事件系统主要是为了解决跨浏览器兼容性问题，并利用事件委托等机制来提升性能。

讲一下 React 的 key 是做什么用的？为什么不推荐用 index 作为 key？

key 的作用：

`key` 是一个特殊的字符串属性，你在创建元素列表时需要包含它。`key` 的主要作用是**帮助 React 识别哪些元素被更改、添加或删除**。它为 React 的 Diff 算法提供了重要的线索。

当一个列表的子元素发生变化时，React 会比较新旧两个列表的子元素。它会根据 `key` 来匹配新旧两个列表中的元素：

- 如果一个 `key` 在新的列表中不存在了，React 会销毁对应的组件。
- 如果一个新的 `key` 在旧的列表中不存在，React 会创建一个新的组件。
- 如果两个 `key` 相同，React 会认为它们是同一个组件，然后比较这两个组件的内容，只更新变化了的部分。

`key` 必须在兄弟元素之间是**唯一的**，但不需要全局唯一。

为什么不推荐用 `index` 作为 `key`？

使用数组的索引（`index`）作为 `key` 在某些情况下会引发严重的问题，尤其是在列表会**重新排序、添加或删除元素**时。

问题场景示例：

假设你有一个列表 `['A', 'B']`，渲染为：

代码块

```
1 <li key={0}>A</li><li key={1}>B</li>
```

现在，如果你在列表的**开头插入一个新元素 C**，列表变为 ['C', 'A', 'B']。如果使用 `index` 作为 `key`，新的渲染结果会是：

代码块

```
1 <li key={0}>C</li><li key={1}>A</li><li key={2}>B</li>
```

React 的 Diff 算法会这样对比：

1. 旧的 `key=0` 的 `<li>` 元素内容是 "A"，新的 `key=0` 的 `<li>` 元素内容是 "C"。React 认为这是**同一个组件**（因为 `key` 相同），于是它会**更新**这个 `<li>` 的内容从 "A" 变成 "C"。
2. 旧的 `key=1` 的 `<li>` 元素内容是 "B"，新的 `key=1` 的 `<li>` 元素内容是 "A"。React 认为这是**同一个组件**，于是它会**更新**这个 `<li>` 的内容从 "B" 变成 "A"。
3. 新的列表中多了一个 `key=2` 的 `<li>`，内容是 "B"。React 会**新增**这个 `<li>`。

这种“就地更新”的方式效率极低，而且如果你的 `<li>` 组件内部有自己的 `state` 或者是非受控的表单元素（如 `<input>`），就会导致 **state 错乱** 的问题。比如，原来属于 "A" 的 `state` 会被 "C" 复用，属于 "B" 的 `state` 会被 "A" 复用。

正确的做法：

应该使用一个**稳定且唯一**的标识来作为 `key`，比如数据项的 `id`。

代码块

```
1 const items = [{id: 'a1', text: 'A'}, {id: 'b2', text: 'B'}];
2 // 在开头插入 C
3 items.unshift({id: 'c3', text: 'C'});
4
5 items.map(item => <li key={item.id}>{item.text}</li>);
```

这样，当在开头插入 "C" 时，React 能够通过 `id` 这个稳定的 `key` 知道，`key='a1'` 和 `key='b2'` 的元素只是位置移动了，而 `key='c3'` 是一个全新的元素。它会执行最高效的操作：**只插入一个新的 `<li>` 元素，而不会去更新已有的元素。**

结论： 只有当列表是静态的，永远不会重新排序或增删时，使用 `index` 作为 `key` 才是安全的。但在绝大多数情况下，都应该使用数据本身提供的稳定且唯一的 ID。

Redux 及状态管理

说说你对 Redux 的理解，它的核心思想和基本原则是什么？中间件的机制是怎样的？

我对 Redux 的理解：

Redux 是一个用于 JavaScript 应用的可预测的状态容器。它不仅仅是一个状态管理库，更是一种应用数据流的架构模式。它将整个应用的 `state` 存储在一个单一的、称为 `store` 的对象树中。当应用的状态需要改变时，我们不能直接修改 `state`，而是必须通过派发（dispatch）一个描述了“发生了什么”的普通对象——`action` 来实现。这个 `action` 会被一个纯函数——`reducer` 接收，`reducer` 会根据 `action` 的类型和内容，计算出新的 `state` 并返回。

核心思想：

Redux 的核心思想是**集中式管理和可预测性**。通过将所有状态集中到一个地方，并规定一套严格的更新流程，使得 `state` 的变化变得可预测、可追溯、易于调试。这对于构建大型、复杂的单页应用（SPA）尤其重要。

三大基本原则：

1. 单一数据源 (Single Source of Truth)：
2. 整个应用的 `state` 被存储在一个单一的 `store` 中的一个对象树里。这使得状态的追踪和调试变得简单，也容易实现服务端渲染和状态持久化。
3. `State` 是只读的 (State is read-only)：
4. 唯一改变 `state` 的方法就是触发 `action`。你不能直接修改 `state` 对象。这保证了所有 `state` 的变化都是集中且有序的，不会有某个视图或网络回调在不知情的情况下随意修改 `state`。
5. 使用纯函数来执行修改 (Changes are made with pure functions)：
6. 为了描述 `action` 如何改变 `state tree`，你需要编写 `reducers`。`Reducer` 是纯函数，它接收前一个 `state` 和一个 `action`，并返回下一个 `state`。因为是纯函数，所以它没有副作用，不修改传入的参数，对于相同的输入，永远有相同的输出。这使得测试变得非常容易，并且也为时间旅行调试等高级功能提供了可能。

中间件（Middleware）的机制：

Redux 中间件提供了一个位于 **dispatching an action** 和 **the moment it reaches the reducer** 之间的第三方扩展点。它允许你封装异步逻辑、日志记录、路由等副作用。

机制：

Redux 的中间件机制本质上是函数式编程中的组合（composition）和高阶函数的应用。
applyMiddleware 函数会将所有的中间件串联起来，形成一个洋葱模型。

1. 当你调用 store.dispatch(action) 时，这个 action 不会立即到达 reducer。
2. 它会先依次穿过你配置的所有中间件。
3. 每个中间件都可以获取到 store 的 getState 和 dispatch 方法，以及当前处理的 action。
4. 中间件可以选择：
 - 直接将 action 传递给下一个中间件（调用 next(action)）。
 - 修改 action，然后传递给下一个中间件。
 - 派发一个新的或多个 action（例如，在异步操作完成后）。
 - 甚至可以阻止这个 action 到达 reducer。
 - 执行一些副作用，比如打印日志、发起 API 请求等。
5. action 像剥洋葱一样，从外到内，一层一层地穿过所有中间件，最后到达原始的 store.dispatch，并最终被 reducer 处理。

这个机制非常强大，像 redux-thunk 这样的中间件，就是通过检查 action 是不是一个函数，如果是，就执行它并传入 dispatch 和 getState，从而让我们能够处理异步逻辑。

除了 Redux，还用过别的状态管理方案吗？比如 MobX, Zustand，它们有什么优缺点？

是的，除了 Redux，我也了解和使用过其他一些状态管理方案。

方案	优点	缺点
MobX	1. 简单直观，上手快： 基于“响应式”和“观察者”模式。你只需要将 state 标记为 observable，当它变化时，所有依赖它的“观察者”（如 React 组件）都会自动更新。模板代码非常少。 2. 细粒度更新，性能好： MobX 通过精确追踪数据依赖，只更新受影响的部分。	1. 不可预测性/魔法： 由于其响应式系统是自动的，背后发生了很多“魔法”。对于复杂的应用，有时很难追踪状态是如何变化的，调试起来可能比 Redux 困难。 2. 自由度高，约束少： 它不像

	据依赖，可以实现非常细粒度的更新，只有真正需要更新的组件才会重新渲染，避免了不必要的渲染。 3. 面向对象友好：使用 class 和装饰器（虽然现在不推荐）的写法非常自然，对习惯了 OOP 的开发者很友好。	Redux 那样有严格的约束（必须通过 action 和 reducer 更新），可以直接修改 state。这虽然方便，但在大型团队中可能会导致代码难以维护和预测。
Zustand	1. 极其简洁：API 非常简单，模板代码极少。创建一个 store 只需要一个简单的 hook。 2. Hooks 原生体验：它的设计完全基于 React Hooks，使用起来就像一个全局的 useState，心智负担很小。 3. 轻量：体积极小，没有太多依赖。 4. 灵活性：它不强制你如何组织代码，可以按模块创建多个 store，也可以轻松集成中间件（如 Redux DevTools）。	1. 功能相对基础：相比 Redux 庞大的生态（各种中间件），Zustand 的开箱即用功能更聚焦于核心的状态管理，一些高级功能可能需要自己实现或寻找第三方库。 2. 社区生态不如 Redux 成熟：虽然非常受欢迎，但 Redux 毕竟发展了很长时间，拥有更庞大的社区和更丰富的解决方案。

总结对比：

- **Redux:** 适合大型、复杂、对状态变更可预测性要求极高的应用。它的学习曲线较陡峭，模板代码多，但换来的是极强的可维护性和调试能力。
- **MobX:** 适合中小型项目或者需要快速开发的项目。它的开发体验非常好，性能也很出色，但需要团队对“响应式”原理有一定理解，并建立良好的代码规范以避免不可预测性。
- **Zustand:** 适合各种规模的项目，尤其是那些推崇 Hooks 并且追求简洁的项目。它在简单性和功能性之间取得了很好的平衡，可以说是现代 React 项目中 Redux 的一个非常强有力的替代品。

JavaScript 的事件循环机制（Event Loop）是什么？讲一下宏任务和微任务。

事件循环（Event Loop） 是 JavaScript 实现异步编程的核心机制。我们知道 JavaScript 是单线程的，意味着它一次只能做一件事。但是，像网络请求、定时器这些耗时操作如果阻塞了主线程，页面就会卡死。事件循环机制就是为了解决这个问题而生的。

事件循环的流程：

1. 所有同步任务都在**主线程**上执行，形成一个**执行栈（Execution Stack）**。
2. 当遇到异步任务时（如 `setTimeout`，`Promise`，AJAX 请求），JavaScript 引擎不会等待它完成，而是将其注册到对应的 **Web APIs**（由浏览器提供）。
3. Web APIs 在异步任务完成后（比如定时器到时、请求返回），会将其**回调函数**放入一个**任务队列（Task Queue）**中。
4. 执行栈中的所有同步任务执行完毕后，主线程会变为空。
5. 事件循环开始工作，它会不断地从任务队列中取出一个回调函数，放到执行栈中去执行。
6. 这个过程不断重复，就形成了事件循环。

宏任务（Macrotask）和微任务（Microtask）：

任务队列实际上被分为了两种：宏任务队列和微任务队列。

• 宏任务 (Macrotask / Task)：

- 代表一个独立的、离散的工作单元。
- 常见的宏任务包括：`script`（整体代码）、`setTimeout`，`setInterval`，`setImmediate` (Node.js), I/O, UI rendering。
- 每次事件循环只会从宏任务队列中**取出一个**任务来执行。

• 微任务 (Microtask / Job)：

- 通常是需要当前任务执行后立即执行的任务，比如对当前操作结果的响应。
- 常见的微任务包括：`Promise.then()`，`Promise.catch()`，`Promise.finally()`，`MutationObserver`，`queueMicrotask()`。
- 在一个宏任务执行结束后，事件循环会检查微任务队列。如果微任务队列不为空，它会**执行所有的**微任务，直到微任务队列清空为止，然后再去执行下一个宏任务。

带有微任务的事件循环流程（更精确的版本）：

1. 执行一个宏任务（比如 `script` 标签里的代码）。
2. 宏任务执行完毕后，检查微任务队列。
3. 如果微任务队列中有任务，就**循环执行所有微任务**，直到微任务队列变空。

4. (可选) 执行 UI 渲染操作。
5. 从宏任务队列中取出下一个宏任务，开始新一轮的事件循环。

示例：

代码块

```
1 console.log('script start');
2
3 setTimeout(function() {
4   console.log('setTimeout');
5 }, 0);
6
7 Promise.resolve().then(function() {
8   console.log('promise1');
9 }).then(function() {
10   console.log('promise2');
11 });
12
13 console.log('script end');
```

输出顺序：

1. script start (宏任务中的同步代码)
2. script end (宏任务中的同步代码)
3. promise1 (当前宏任务结束，执行所有微任务)
4. promise2 (上一个微任务产生的新的微任务，继续执行)
5. setTimeout (微任务队列清空，执行下一个宏任务)

this 的指向问题，箭头函数和普通函数的 this 有什么不同？如何改变 this 的指向？

this 是 JavaScript 中的一个关键字，它的值在函数被调用时确定，而不是在函数定义时确定。

this 的指向非常灵活，也因此成为一个常见的难点。

普通函数的 this 指向规则：

1. 全局上下文：在全局作用域中，this 指向全局对象 (window in browsers, global in Node.js)。
2. 作为对象方法调用：当函数作为对象的一个方法被调用时，this 指向调用该方法的对象。
obj.func()，this 就是 obj。

- 3. 作为普通函数调用：当函数被直接调用时（不是作为对象的方法），在非严格模式下，`this` 指向全局对象 (`window`)；在严格模式 (`'use strict'`) 下，`this` 是 `undefined`。
- 4. 构造函数中：当使用 `new` 关键字调用一个函数时，这个函数就作为构造函数，`this` 会指向一个新创建的空对象。
- 5. DOM 事件处理函数：在 HTML 事件处理函数中，`this` 通常指向触发事件的那个元素。

箭头函数 (Arrow Function) 的 `this`：

箭头函数是 ES6 中引入的，它没有自己的 `this` 绑定。箭头函数的 `this` 是词法作用域的，意味着它会捕获其定义时所在上下文的 `this` 值。一旦确定，就无法再改变。

主要区别总结：

特性	普通函数	箭头函数
this 的绑定	调用时动态确定。	定义时词法确定（继承外层作用域的 this）。
arguments 对象	有自己的 arguments 对象。	没有自己的 arguments 对象，会继承外层作用域的。
作为构造函数	可以使用 new 关键字。	不能使用 new，会抛出错误。
call, apply, bind	可以用来改变 this 的指向。	call, apply, bind 可以调用函数，但无法改变其 this 的指向。

如何改变 `this` 的指向？

对于普通函数，主要有三种方法可以显式地改变 `this` 的指向：

- 1. `Function.prototype.call(thisArg, arg1, arg2, ...)`：
 - 立即调用该函数。
 - 将函数的 `this` 指向第一个参数 `thisArg`。
 - 其余参数以逗号分隔的形式传递给函数。

代码块

```
1  function greet() {
2    console.log(this.name);
3  }
4  const person = { name: 'Gemini' };
5  greet.call(person); // 输出: Gemini
```

1. `Function.prototype.apply(thisArg, [argsArray])`:

- 立即调用该函数。
- 与 `call` 类似，但它接收一个包含所有参数的**数组**（或类数组对象）作为第二个参数。

代码块

```
1 function sum(a, b) {  
2   console.log(this.label + (a + b));  
3 }  
4 const calc = { label: 'Result: ' };  
5 sum.apply(calc, [10, 20]); // 输出: Result: 30
```

1. `Function.prototype.bind(thisArg, arg1, arg2, ...)`:

- **不会立即调用函数。**
- 它会**创建一个新的函数**，这个新函数的 `this` 被永久地绑定到 `thisArg`。
- 可以预先设置部分参数（柯里化）。

代码块

```
1 function logMessage(level, message) {  
2   console.log(`[${level}] ${this.source}: ${message}`);  
3 }  
4 const logger = { source: 'System' };  
5 const systemLog = logMessage.bind(logger, 'INFO'); // 创建一个新函数，并预设了  
   this 和第一个参数  
6  
7 systemLog('Application started.');// 输出: [INFO] System: Application started.
```

网络与浏览器

讲一下从输入 URL 到页面加载完成，整个过程发生了什么？

从输入 URL 到页面加载完成是一个非常复杂但有序的过程，可以概括为以下几个主要步骤：

1. URL 解析 (Parsing):

- 浏览器首先会解析用户输入的 URL，判断其合法性，并提取出协议（`http`，`https`）、域名（`www.google.com`）、端口（默认为 80 或 443）、路径（`/search`）等信息。

2. DNS 查询 (Domain Name System):

- 浏览器需要将域名转换为服务器的 IP 地址才能建立连接。
- 查询顺序：浏览器缓存 -> 操作系统缓存 -> Hosts 文件 -> 本地 DNS 服务器 -> 根 DNS 服务器 -> ... (递归或迭代查询)，直到找到对应的 IP 地址。

3. 建立 TCP 连接:

- 浏览器获得服务器 IP 地址后，会与服务器建立一个 TCP 连接。这个过程就是著名的三次握手。
- 如果 URL 是 `https` 协议，还需要在 TCP 连接之上建立一个 TLS/SSL 连接，进行加密通信。

4. 发送 HTTP 请求:

- TCP 连接建立后，浏览器会构造一个 HTTP 请求报文，并通过该连接发送给服务器。
- 请求报文包括：请求行（请求方法 GET/POST，URL，HTTP 版本）、请求头（Host, User-Agent, Cookie, Accept 等）和请求体（POST 请求时携带的数据）。

5. 服务器处理请求并响应:

- 服务器接收到请求后，会根据 URL 和参数进行处理（例如查询数据库、执行业务逻辑）。
- 处理完成后，服务器会构造一个 HTTP 响应报文并发送回浏览器。
- 响应报文包括：状态行（HTTP 版本，状态码如 200 OK, 404 Not Found）、响应头（Content-Type, Set-Cookie, Cache-Control 等）和响应体（通常是 HTML 内容）。

6. 浏览器接收响应并解析:

- 浏览器接收到服务器的响应。首先会检查状态码。如果是 200，则开始解析响应体。
- 浏览器可能会根据响应头中的 `Cache-Control` 或 `Expires` 等字段判断是否要缓存这个资源。

7. 渲染页面:

- 这是最复杂的一步，可以细分为：
 - **构建 DOM 树**：浏览器解析 HTML 文档，将标签转换为一个树形结构的 DOM (Document Object Model)。
 - **构建 CSSOM 树**：同时，浏览器会解析 CSS 文件（包括外部 CSS、内联样式和 `<style>` 标签），构建 CSSOM (CSS Object Model)。
 - **构建渲染树 (Render Tree)**：将 DOM 树和 CSSOM 树结合起来，生成渲染树。渲染树只包含需要显示的节点及其样式信息（比如 `display:none` 的节点就不会在渲染树中）。
 - **布局 (Layout/Reflow)**：根据渲染树，计算出每个节点在屏幕上的确切位置和大小。

- **绘制 (Painting)**: 调用 GPU, 将各个节点绘制到屏幕上。

- 在解析 HTML 的过程中, 如果遇到 `<img>`, `<link rel="stylesheet">`, `<script>` 等标签, 浏览器会**并行地**发起新的 HTTP 请求去获取这些资源。**需要注意的是**, `<script>` 标签 (特别是没有 `async` 或 `defer` 属性的) 会阻塞 DOM 的解析, 直到脚本下载并执行完

8. 连接关闭:

- 当所有数据都传输完毕后, 通常会通过**四次挥手**来关闭 TCP 连接。

这个过程完成后, 用户就能看到完整的页面了。

HTTP 缓存有哪几种? 分别是什么机制?

HTTP 缓存是 Web 性能优化的关键手段。它主要分为两大类: **强缓存**和**协商缓存**。

9. 强缓存 (Strong Cache)

- **机制**: 浏览器在请求一个资源时, 会先检查本地缓存。如果缓存未过期, 就**直接使用缓存的副本**, **不会向服务器发送任何请求**。HTTP 状态码通常是 `200 OK (from memory cache / from disk cache)`。
- **控制字段 (Response Headers)**:
 - **Expires (HTTP/1.0)**: 一个绝对时间的 GMT 格式字符串, 例如 `Expires: Wed, 21 Oct 2025 07:28:00 GMT`。它告诉浏览器缓存的过期时间。**缺点是**依赖于客户端本地时间, 如果客户端时间不准, 可能会导致缓存失效。
 - **Cache-Control (HTTP/1.1)**: 一个相对时间的指令, 优先级高于 `Expires`。它更灵活、功能更强大。常用值包括:
 - `max-age=<seconds>`: 缓存的有效时间, 单位是秒。例如 `Cache-Control: max-age=3600` 表示资源在 1 小时内有效。
 - `public`: 响应可以被任何缓存 (如 CDN、代理服务器) 缓存。
 - `private`: 响应只能被最终用户的浏览器缓存。
 - `no-cache`: **需要进行协商缓存**。浏览器会使用缓存, 但每次使用前都必须向服务器确认资源是否仍然有效。
 - `no-store`: **禁止缓存**。浏览器和任何中间缓存都不能存储这个响应的任何部分。

10. 协商缓存 (Negotiation Cache / Conditional Request)

- **机制**: 当强缓存失效后 (或者 `Cache-Control` 设置为 `no-cache`), 浏览器会向服务器发送一个**条件请求**。服务器会根据请求头中的信息判断资源是否有更新。
 - 如果**没有更新**, 服务器返回 `304 Not Modified` 状态码, 响应体为空。浏览器继续使用本地的缓存副本。

- 如果有更新，服务器返回 `200 OK` 状态码，并在响应体中携带最新的资源内容。浏览器使用新资源并更新本地缓存。
- 控制字段 (成对出现):
 - `Last-Modified` / `If-Modified-Since` :
 - 服务器在首次响应时，通过 `Last-Modified` 响应头告诉浏览器资源的最后修改时间。
 - 浏览器在下一次请求时，通过 `If-Modified-Since` 请求头将这个时间发回给服务器。
 - 服务器比较这个时间和资源的实际最后修改时间。如果一致，返回 304。
 - 缺点：时间戳只能精确到秒，如果在 1 秒内文件被多次修改，无法检测到；某些服务器上的文件时间戳可能会改变，但内容没变。
 - `ETag` / `If-None-Match` (优先级更高):
 - `ETag` (Entity Tag) 是服务器为资源生成的唯一标识符（类似文件指纹或哈希值）。只要资源内容改变，`ETag` 就会改变。
 - 服务器在首次响应时，通过 `ETag` 响应头将这个标识符发给浏览器。
 - 浏览器在下一次请求时，通过 `If-None-Match` 请求头将这个 `ETag` 发回给服务器。
 - 服务器比较浏览器发来的 `ETag` 和当前资源的 `ETag`。如果一致，返回 304。
 - `ETag` 解决了 `Last-Modified` 的缺点，能更精确地判断资源是否变化。

缓存流程总结：

1. 浏览器发起请求。
2. 检查本地缓存，查看强缓存是否命中（检查 `Cache-Control` 的 `max-age` 和 `Expires`）。
3. 如果强缓存命中，直接使用缓存，流程结束。
4. 如果强缓存未命中，发起 HTTP 请求，并带上协商缓存的头信息（`If-Modified-Since` 或 `If-None-Match`）。
5. 服务器根据协商缓存头信息判断资源是否有更新。
6. 如果资源未更新，返回 304，浏览器使用本地缓存。
7. 如果资源已更新，返回 200 和最新的资源内容，浏览器使用新资源并更新缓存。

TCP 的三次握手和四次挥手过程能说一下吗？

TCP (Transmission Control Protocol) 是一种面向连接的、可靠的、基于字节流的传输层通信协议。它的连接建立（三次握手）和连接断开（四次挥手）是保证其可靠性的重要机制。

三次握手 (Three-Way Handshake) - 建立连接

三次握手的目的是同步客户端和服务器的序列号（Sequence Number, SEQ）和确认号（Acknowledgement Number, ACK），并交换 TCP 窗口大小等信息。

- **第一次握手 (SYN):**

- **客户端 -> 服务器**
- 客户端想要建立连接，向服务器发送一个 `SYN` (Synchronize Sequence Numbers) 报文段。
- 在这个报文段中，客户端会随机选择一个初始序列号 `seq = x`。
- 此时，客户端进入 `SYN_SENT` 状态。

- **第二次握手 (SYN + ACK):**

- **服务器 -> 客户端**
- 服务器收到客户端的 `SYN` 报文后，如果同意建立连接，会回复一个报文段。
- 这个报文段同时包含 `SYN` 和 `ACK` (Acknowledgement) 标志。
- 服务器也会随机选择自己的一个初始序列号 `seq = y`。
- 同时，它会将确认号设置为客户端序列号加一，即 `ack = x + 1`，表示已收到客户端的 `SYN`。
- 此时，服务器进入 `SYN_RCVD` 状态。

- **第三次握手 (ACK):**

- **客户端 -> 服务器**
- 客户端收到服务器的 `SYN+ACK` 报文后，会向服务器发送一个 `ACK` 报文段。
- 这个报文段的序列号是 `seq = x + 1`。
- 确认号是服务器序列号加一，即 `ack = y + 1`，表示已收到服务器的 `SYN`。
- 此报文发送完毕后，客户端和服务器都进入 `ESTABLISHED` 状态，TCP 连接成功建立，可以开始传输数据。

四次挥手 (Four-Way Handshake) - 断开连接

由于 TCP 是全双工的，连接的每一方都必须独立地关闭自己的发送通道。因此，断开连接需要四步。

- **第一次挥手 (FIN):**

- **客户端 -> 服务器**
- 客户端数据发送完毕，希望断开连接，向服务器发送一个 `FIN` (Finish) 报文段。
- 报文段中包含一个序列号 `seq = u`。
- 此时，客户端进入 `FIN_WAIT_1` 状态。客户端不能再发送数据，但仍可以接收数据。

- **第二次挥手 (ACK):**

- **服务器 -> 客户端**

- 服务器收到客户端的 `FIN` 报文后，回复一个 `ACK` 报文段。
 - 确认号为 `ack = u + 1`。
 - 此时，服务器进入 `CLOSE_WAIT` 状态。这个状态下，服务器到客户端的连接还未关闭，服务器仍然可以向客户端发送数据。客户端收到这个 `ACK` 后，进入 `FIN_WAIT_2` 状态。
 - **第三次挥手 (FIN):**
 - **服务器 -> 客户端**
 - 服务器数据也发送完毕，准备关闭连接，向客户端发送一个 `FIN` 报文段。
 - 报文段中包含一个序列号 `seq = w`。
 - 此时，服务器进入 `LAST_ACK` 状态，等待客户端的最后确认。
 - **第四次挥手 (ACK):**
 - **客户端 -> 服务器**
 - 客户端收到服务器的 `FIN` 报文后，回复一个 `ACK` 报文段。
 - 确认号为 `ack = w + 1`。
 - 此时，客户端进入 `TIME_WAIT` 状态。需要等待 $2 * \text{MSL}$ (Maximum Segment Lifetime, 报文最大生存时间)，以确保服务器收到了这个 `ACK`，并防止已失效的连接请求报文段出现在本连接中。
 - 服务器收到 `ACK` 后，立即进入 `CLOSED` 状态。
 - 客户端在等待 $2 * \text{MSL}$ 后，如果没有收到服务器的重传请求，也进入 `CLOSED` 状态。连接正式断开。
-

CSS 与构建工具

CSS 的 Flex 布局 and Grid 布局你用过吗？分别适合什么场景？

是的，Flex 和 Grid 布局我都非常熟悉，它们是现代 CSS 布局的基石，解决了传统布局方式（如 float, position）的很多痛点。

Flex 布局 (Flexible Box Layout)

- **核心思想：一维布局。** 它让你能够在一个容器里，沿着一条主轴（水平或垂直）对子元素进行对齐、分布和排序。

- **关键概念：** 主轴 (main axis)、交叉轴 (cross axis)、`flex-direction`，`justify-content`，`align-items`，`flex-grow`，`flex-shrink`，`flex-basis`。
- **适合场景：**
 - **组件内部的元素对齐：** 比如按钮内的图标和文字的对齐、导航栏菜单项的均匀分布、卡片头部的标题和操作按钮的对齐。
 - **线性列表布局：** 任何形式的单行或单列列表，如文章列表、商品列表。
 - **空间的均匀分配：** 当你希望几个元素能自动填满一行或一列，并能按比例分配空间时，Flex 非常好用。
 - **垂直居中：** 使用 `align-items: center` 和 `justify-content: center` 可以非常简单地实现元素的垂直和水平居中，这是传统 CSS 的一个痛点。

Grid 布局 (Grid Layout)

- **核心思想：二维布局。** 它将一个容器划分为行和列，形成一个网格，然后你可以将子元素精确地放置在网格的指定位置。它同时控制着水平和垂直两个方向的布局。
- **关键概念：** 网格容器 (grid container)、网格项 (grid item)、网格线 (grid line)、网格轨道 (grid track)、网格单元 (grid cell)、`grid-template-columns`，`grid-template-rows`，`grid-gap`，`grid-column`，`grid-row`。
- **适合场景：**
 - **整个页面的宏观布局：** 比如传统的页眉、页脚、侧边栏、主内容区的布局，Grid 可以非常清晰和稳定地实现。
 - **复杂的、非线性的二维布局：** 当元素的对齐关系不仅仅是一条线，而是需要在两个维度上都对齐时，Grid 是不二之选。例如，图片画廊、棋盘布局、仪表盘 (Dashboard)。
 - **内容与布局分离：** Grid 允许你使用网格线来定位元素，这意味着你可以在不改变 HTML 结构的情况下，仅通过修改 CSS 就能彻底改变元素的布局位置，非常灵活。

总结与对比：

- **Flex 是一维的，Grid 是二维的。** 这是最核心的区别。
- **Flex 是“内容驱动”的**，它根据内容的大小来调整布局。**Grid 是“布局驱动”的**，你先定义好网格，再把内容放进去。
- **“用 Flex 还是用 Grid？”** 很多时候不是一个非此即彼的问题，它们可以**组合使用**。一个常见的模式是：**用 Grid 做整个页面的宏观布局，然后在大布局的某个区域内部，使用 Flex 来对齐和分布其中的小组件。**

Webpack 和 Vite 的区别是什么？Vite 为什么快？

Webpack 和 Vite 都是现代前端项目中最主流的构建工具，但它们的底层原理和开发体验有很大不同。

特性	Webpack	Vite
开发服务器启动	慢。 在启动时，Webpack 会从入口文件开始，将所有模块（JS, CSS, 图片等）打包（bundle）成一个或多个文件，然后再启动开发服务器。项目越大，启动越慢。	极快。 Vite 利用了现代浏览器对原生 ES Module 的支持。它在启动时不进行打包，而是直接启动一个开发服务器。当浏览器请求某个模块时，Vite 的服务器才会按需编译和提供这个模块。
热更新 (HMR)	相对较慢。 当一个文件被修改时，Webpack 通常需要重新构建这个模块以及与之相关的依赖，然后将更新推送到浏览器。	极快。 Vite 的 HMR 也是基于原生 ESM。当一个文件被修改时，Vite 只需要精确地让浏览器重新请求这一个被修改的模块，然后替换它。这个过程与项目的大小无关，所以速度非常快。
打包原理	Bundle-based 。核心思想是“一切皆模块”，将所有资源打包在一起。	Native ESM-based (dev) + Rollup (build) 。开发环境利用原生 ES Module，生产环境则使用 Rollup 进行打包优化。
配置	复杂。 需要通过大量的 loader 和 plugin 来配置如何处理不同类型的文件，配置项繁多，学习成本高。	开箱即用。 配置非常简洁，大部分常见需求都已内置，对 TypeScript, JSX, CSS 等都有原生支持。

Vite 为什么快？

Vite 的快主要体现在**开发阶段**，其核心原因有两点：

1. 利用原生 ES Module (ESM) 实现 No-Bundle 开发服务：

- 传统的 Webpack 会在启动时将所有模块打包成一个或几个大的 bundle 文件。这个“打包”过程本身就很耗时，并且随着项目规模的增长，时间会越来越长。

- Vite 反其道而行之。它启动一个开发服务器，然后直接将你的源码（如 `.js`，`.vue`，`.jsx` 文件）交给浏览器。现代浏览器本身就支持 `<script type="module">`，可以解析 `import` 和 `export` 语法，并能自己发起 HTTP 请求去加载被 `import` 的其他模块。
- 当浏览器请求一个模块时，Vite 的开发服务器会拦截这个请求，**按需编译**（例如将 TS 转成 JS，将 JSX 转成 JS），然后返回给浏览器。这个编译是即时的、按需的，而不是像 Webpack 那样“全量”的。

2. 基于 esbuild 的超快编译：

- 对于 TypeScript、JSX 等需要预编译的语言，Vite 使用了 **esbuild**。
- esbuild 是一个用 Go 语言编写的 JavaScript 打包器和编译器，它的速度比用 JavaScript 编写的传统工具（如 Babel, tsc）快 **10-100 倍**。
- Go 语言是编译型语言，并且充分利用了多核心 CPU 来进行并行处理，这是其速度远超 Node.js 工具的主要原因。Vite 利用 esbuild 进行预构建依赖和 TS/JSX 的转换，极大地提升了开发服务器的响应速度和依赖预构建的速度。

生产环境构建：

需要注意的是，Vite 的“No-Bundle”策略主要用于开发环境。在**生产环境**，为了获得最佳的加载性能（减少 HTTP 请求次数、进行代码压缩和 Tree-shaking），Vite 仍然会进行打包。它使用 **Rollup** 作为其生产环境的打包工具，因为 Rollup 在代码分割和 Tree-shaking 方面做得非常出色，能生成高度优化的代码。

你在项目中做过哪些 Webpack 的配置和优化？

在过去的项目中，为了提升开发体验和生产环境的性能，我做过一系列的 Webpack 配置和优化，主要可以分为**提升打包速度**和**优化产出代码**两大类。

一、提升打包速度（优化开发体验）

1. **升级版本**：确保 Webpack、Node.js 以及相关的 loader、plugin 保持在较新的稳定版本，新版本通常会带来性能上的改进。
2. **缩小构建范围 (exclude/include)**：在使用 `babel-loader` 等转换工具时，明确使用 `include` 指定要处理的源码目录（通常是 `src`），并用 `exclude` 排除掉 `node_modules`。这是最基本也是最有效的优化。

3. 缓存 (Caching)：

- 配置 `cache: { type: 'filesystem' }`。Webpack 5 默认开启持久化缓存，可以将模块的编译结果缓存到文件系统中。第二次启动构建时，如果文件未改变，会直接使用缓存，大大加快启动和热更新速度。
- 为 `babel-loader` 开启 `cacheDirectory: true`。

4. **多进程打包 (Thread-loader / HappyPack)**: 对于计算量大的 loader (如 `babel-loader`), 可以使用 `thread-loader` 将其放置在一个独立的 worker 池中运行, 利用多核 CPU 的能力并行处理。
5. **优化 `resolve` 配置**:
 - 合理配置 `resolve.extensions`, 减少不必要的文件后缀匹配。将常用后缀放在前面。
 - 配置 `resolve.alias`, 为常用的模块路径创建别名, 可以减少 Webpack 的模块搜索路径。
6. **使用 `DllPlugin` 和 `DllReferencePlugin`**: 对于不经常变更的第三方库 (如 React, Vue 全家桶), 可以将它们预先打包成一个动态链接库 (DLL)。在日常开发中, Webpack 就无需再重复打包这些库, 只需引用即可, 可以极大地提升开发时的构建速度。

二、优化产出代码 (提升加载性能)

1. 代码压缩:

- 使用 `TerserWebpackPlugin` 压缩 JavaScript 代码 (Webpack 5 已内置)。
- 使用 `CssMinimizerWebpackPlugin` 压缩 CSS 代码。

2. 代码分割 (Code Splitting):

- **入口分割**: 通过配置多个 `entry`, 实现多页应用 (MPA) 的打包。
- **动态导入**: 使用 `import()` 语法, 对路由组件或大型模块进行懒加载, 按需加载代码, 减小首屏加载体积。
- **抽取公共代码**: 配置 `optimization.splitChunks`, 将 `node_modules` 中的公共库 (如 React) 和业务代码中的公共模块抽取成单独的 chunk 文件, 利用浏览器缓存。

3. Tree Shaking:

- 确保开启 `mode: 'production'`, Webpack 会自动启用 Tree Shaking。
- 确保使用的第三方库支持 Tree Shaking (即它们的 `package.json` 中有 `sideEffects` 字段或使用的是 ES Module 导出)。
- 在自己的代码中, 始终使用 `import/export` (ESM) 语法, 而不是 `require` (CommonJS)。

4. 分析打包体积 (Bundle Analysis):

- 使用 `webpack-bundle-analyzer` 插件。它可以生成一个可视化的报告, 清晰地展示出打包结果中各个模块的大小占比, 帮助我们找到可以优化的“大块头”模块。

5. 合理使用 `source-map`:

- 开发环境使用 `eval-cheap-module-source-map`, 构建速度快, 调试信息相对完整。
- 生产环境使用 `source-map` (上传到错误监控平台) 或 `none` (如果不需要线上调试)。

通过以上这些综合手段, 可以显著改善 Webpack 的构建性能和最终产出代码的质量。

算法题

算法题：实现一个 debounce (防抖) 函数。

好的。防抖 (debounce) 的核心思想是：**在事件被触发 n 秒后再执行回调，如果在这 n 秒内又被触发，则重新计时。** 这非常适合用于处理像输入框搜索、窗口大小调整等频繁触发的事件。

下面是我实现的 `debounce` 函数，包含了一些进阶功能的考虑，比如立即执行选项、取消功能和 `this` 指向及参数的正确传递。

代码块

```
1  /**
2   * 实现一个防抖函数
3   * @param {Function} func 需要执行的函数
4   * @param {number} wait 延迟执行的毫秒数
5   * @param {boolean} immediate 是否立即执行一次。true 表示首次触发时立即执行，之后 n
   秒内不再触发。
6   * @returns {Function} 返回一个经过防抖处理的函数
7   */function debounce(func, wait, immediate) {
8     let timeout; // 定时器变量let result; // 存储立即执行时的返回值// 返回
   一个新的函数，这个函数将作为事件处理程序const debounced = function(...args) {
9       // 保存 this 指向和事件参数const context = this;
10
11       // 如果之前有定时器，则清除它，重新开始计时if (timeout) {
12         clearTimeout(timeout);
13       }
14
15       if (immediate) {
16         // 如果设置了立即执行// callNow 变量用于判断是否是首次触发const callNow =
!timeout;
17         // 设置定时器，在 wait 时间后将 timeout 清空// 这样在 wait 时间段过后，又可以立
   即执行一次了
18         timeout = setTimeout(() => {
19           timeout = null;
20         }, wait);
21
22         // 如果是首次触发，则立即执行 funcif (callNow) {
23           result = func.apply(context, args);
24         }
25       } else {
26         // 如果不是立即执行，则设置一个新的定时器// 在 wait 时间后执行 func
```

```

27     timeout = setTimeout(() => {
28         func.apply(context, args);
29     }, wait);
30 }
31
32 // 如果是立即执行, 返回上一次执行的结果return result;
33 };
34
35 // 添加一个取消方法
36 debounced.cancel = function() {
37     clearTimeout(timeout);
38     timeout = null;
39 };
40
41 return debounced;
42 }
43
44 // --- 使用示例 ---// 1. 基础用法 (非立即执行)const handleMouseMove = (e) => {
45     console.log('Mouse is moving:', e.clientX, e.clientY);
46 };
47
48 const debouncedMouseMove = debounce(handleMouseMove, 500);
49
50 // 模拟mousemove事件绑定// document.addEventListener('mousemove',
51 // debouncedMouseMove);// 你会发现, 只有当你停止移动鼠标半秒后, 控制台才会打印信息// 2.
52 // 立即执行用法const handleClick = () => {
53     console.log('Button clicked immediately!');
54 };
55
56 const debouncedClick = debounce(handleClick, 1000, true);
57
58 // 模拟按钮点击// button.addEventListener('click', debouncedClick);// 第一次点击会
59 // 立即打印, 之后1秒内的所有点击都会被忽略// 3. 取消功能const handleInput = (e) => {
60     console.log('Input value:', e.target.value);
61 };
62
63 const debouncedInput = debounce(handleInput, 500);
64
65 // 模拟输入// inputElement.addEventListener('input', debouncedInput);// 假设在某
66 // 个时刻我们想取消还未执行的回调// debouncedInput.cancel();

```

代码讲解:

- 闭包:** 整个函数利用了闭包的特性。 `timeout` 和 `result` 变量被保存在返回的 `debounced` 函数的闭包中, 使得每次调用 `debounced` 函数时都能访问和修改同一个 `timeout` 变量。

2. **this 和 arguments**：使用 `const context = this` 和 `...args` 来捕获 `debounced` 函数被调用时的 `this` 上下文和所有参数，然后通过 `func.apply(context, args)` 将它们正确地传递给原始的 `func` 函数。
3. **核心逻辑 (非立即执行)**：每次触发 `debounced` 函数时，先 `clearTimeout(timeout)` 清除掉上一个定时器，然后设置一个新的定时器。这样就保证了只有最后一次触发后的 `wait` 时间内没有新的触发，`func` 才会执行。
4. **立即执行 (`immediate: true`)**：
 - 通过 `callNow = !timeout` 来判断是否是“冷却期”内的第一次触发。如果是，则立即执行 `func`。
 - 同时，依然会设置一个定时器，它的作用不是执行 `func`，而是在 `wait` 毫秒后将 `timeout` 清空，相当于“重置冷却时间”，允许下一次的立即执行。
5. **取消 (`cancel` 方法)**：在返回的 `debounced` 函数上挂载一个 `cancel` 方法，该方法可以直接清除当前的定时器，从而阻止待执行的 `func` 被调用。这是一个很实用的扩展功能。

二面参考答案：

React 的 Fiber 架构是什么？它解决了什么问题？

React Fiber 是什么？

React Fiber 是 React 16 版本引入的一套全新的**协调 (Reconciliation) 引擎**。它不是指某个具体的功能，而是对 React 核心算法的重写。从外部看，React 的 API 几乎没有变化，但其内部的工作方式发生了根本性的改变。

我们可以把 Fiber 理解为一种数据结构，它是一个**带有额外信息的 JavaScript 对象**。每一个 React 元素 (Element) 在运行时都会对应一个 Fiber 节点。这些 Fiber 节点通过 `child`、`sibling` 和 `return` 指针，构成了一个**链表树**，即 Fiber 树。这个树结构模拟了组件的调用栈，使得 React 可以手动地管理和调度这个“虚拟的调用栈”。

Fiber 解决了什么问题？

Fiber 架构主要解决了**主线程长时间被占用导致页面卡顿**的问题。

在 Fiber 之前的版本 (React 15)，React 的协调过程是**同步的、递归的**。当一个组件的状态发生变化，React 会从根节点开始，递归地遍历整个组件树，找出所有变化的节点，然后一次性地更新到 DOM 上。这个过程被称为“Stack Reconciler”。

这种方式的**致命缺点是：一旦开始，就无法中断**。如果组件树非常庞大和复杂，这个递归对比 (Diffing) 的过程会非常耗时，它会长时间占用 JavaScript 主线程。在这期间，浏览器无法响应用户的任何操作 (如点击、滚动)，也无法执行动画等高优先级的任务，从而导致页面卡顿、掉帧，给用户带来非常糟糕的体验。

为了解决这个问题，Fiber 架构引入了两个核心概念：

1. 可中断与恢复 (Interruptible and Resumable):

- Fiber 将原来“一气呵成”的更新任务，拆分成了一个个小的**工作单元 (Unit of Work)**。每个 Fiber 节点就是一个工作单元。
- React 会在一个循环（`requestIdleCallback` 或 `scheduler`）中处理这些工作单元。每完成一个单元的工作后，它会检查是否有更高优先级的任务（比如用户输入）。
- 如果有，React 会**暂停**当前的更新过程，让出主线程去执行高优先级任务。
- 当主线程空闲下来后，React 会从上次暂停的地方**恢复**并继续构建 Fiber 树。

2. 任务优先级 (Priority):

- Fiber 架构为不同的更新任务赋予了优先级。例如，由用户输入（如打字）触发的更新，其优先级就高于由网络请求返回数据触发的更新。
- React 的调度器 (Scheduler) 会根据优先级来决定先执行哪个更新任务，确保高优先级的任务能够得到及时的响应。

总结：Fiber 架构的核心目标是实现**异步可中断的更新**。它通过将渲染/更新过程碎片化，使得 React 可以在渲染一半时暂停，去处理更紧急的任务，从而极大地提升了复杂应用的响应速度和用户体验。

React 的 Diff 算法是怎么工作的？和 Vue 的 Diff 算法有什么区别？

React 的 Diff 算法，即协调 (Reconciliation) 过程中的核心部分，是用来比较新旧两棵虚拟 DOM (Virtual DOM) 树，找出最小化的变更，然后更新到真实 DOM 上的。

React Diff 算法的工作方式：

React 的 Diff 算法基于两个核心前提，使得其复杂度从 $O(n^3)$ 优化到了 $O(n)$ ：

1. **两个不同类型的元素会产生出不同的树。** 如果根节点的元素类型不同（比如 `<div>` 变成了 `<span>`），React 会直接销毁旧的树，创建一棵全新的树。
2. **开发者可以通过 `key` 属性，来暗示哪些子元素在不同渲染下能保持稳定。**

基于这两个前提，React 的 Diff 算法主要分三层进行：

1. Tree Diff (树的比较):

- React 对两棵树进行**同层比较**，不会跨层级移动节点。
- 如果根节点发生变化（类型不同），则直接卸载整棵旧树，创建新树。这个策略非常高效，因为跨层级的 DOM 操作通常很少，而且会带来非常复杂的组件状态转移问题。

2. Component Diff (组件的比较):

- 如果比较的是两个相同类型的组件，React 会保留组件实例，更新 `props`，并调用组件的生命周期方法（或 `useEffect`）。
- 如果是不同类型的组件，则会被当作根节点变化处理，直接销毁旧组件，创建新组件。

3. Element Diff (元素的比较):

- 这是 Diff 算法最核心、最复杂的部分，用于比较同一层级下的一组子节点。
- **没有 `key` 的情况**：React 会逐个对比新旧列表中的节点。如果发现不一致，比如旧的是 `div` 新的是 `p`，它会从这个位置开始，销毁后面所有的旧节点，然后创建所有新的节点。这种方式在列表发生**排序或插入**操作时，效率极低。
- **有 `key` 的情况**：React 会使用 `key` 来匹配新旧列表中的节点。它会遍历新列表，通过 `key` 去旧列表中寻找**可复用的**节点。
 - **更新 (UPDATE)**：找到 `key` 相同但内容不同的节点，进行更新。
 - **移动 (MOVE)**：找到 `key` 相同但位置不同的节点，进行移动操作。
 - **创建 (CREATE)**：在新列表中存在，但在旧列表中不存在的 `key`，创建新节点。
 - **删除 (DELETE)**：在旧列表中存在，但在新列表中不存在的 `key`，删除旧节点。

和 Vue 的 Diff 算法的区别：

React 和 Vue 的 Diff 算法在核心思想上是类似的（都基于 VDOM 和 `key`），但具体的实现策略上存在显著差异。

特性	React Diff	Vue Diff
比较策略	单端比较： React 遍历新列表，用 key 在旧列表中查找。它会记录一个 lastPlacedIndex 索引，来判断一个节点是否需要移动。如果一个节点在旧列表中的索引小于 lastPlacedIndex，则认为它需要移动。	双端比较 (Double-Ended): Vue 的 Diff 算法（在 Vue 2 中非常经典，Vue 3 有所优化但思想类似）同时维护四个指针： oldStartIdx, oldEndIdx, newStartIdx, newEndIdx。它会优先尝试比较 新头 vs 旧头、新尾 vs 旧尾、新头 vs 旧头、新头 vs 旧尾 这四种情况。
优化方向	React 的策略更偏向于追加操作。当新节点大部分都是在列表末尾添加时，其效率非常高，因为不需要移动。	Vue 的双端比较策略，对列表的反转、首尾元素的移动等场景做了特别优化，在这类场景下，它可以找到最优的移动路径，减少 DOM 操作次数。
底层实现	React 的 Diff 是 Fiber 架构的一部分，是可中断的。	Vue 2 的 Diff 是同步的。Vue 3 引入了静态标记（Patch Flags），在编译阶段就分析出哪些节点是动态的，从而在 Diff 时跳过所有静态节点，只对比动态部分，进一步提升了性能。
总结	React 认为 Web UI 中跨层级的移动操作很少，同层级的移动主要是追加，所以采用了相对简单的单端比较策略。	Vue 则试图通过更复杂的双端比较策略，来覆盖更多的移动场景（如反转），尽可能地减少 DOM 操作。

可以说，Vue 的 Diff 算法在某些特定场景（如列表反转）下会比 React 更高效，而 React 的策略则更通用和简单。

如何对一个 React 应用进行性能优化？从你的项目实践出发，具体谈谈。

在我之前的项目中，React 应用的性能优化是一个持续进行的过程，我会从**代码层面**、**构建层面**和**监控层面**三个维度来系统地进行。

一、代码层面（开发阶段）

这是最核心也是最常见的优化部分。

1. PureComponent, React.memo, 和 shouldComponentUpdate:

- **实践**：在项目中，我们有一个数据展示的仪表盘页面，里面有大量的图表和卡片组件。当父组件的一个不相关的 state 变化时，会导致所有子组件都重新渲染，造成明显卡顿。
- **解决方案**：我们将所有纯展示型的子组件（即 props 不变，UI 就不变的组件）都用 `React.memo` 包裹起来。`React.memo` 会对 props 进行浅比较，只有 props 发生变化时，组件才会重新渲染。对于 Class Component，则使用 `PureComponent`。这显著减少了不必要的 `re-render` 次数。

2. useCallback 和 useMemo:

- **实践**：在一个复杂的表单页面，一个父组件定义了多个事件处理函数（如 `handleInputChange`，`handleSubmit`），并将它们传递给了多个子组件（这些子组件已经被 `React.memo` 包裹）。但我们发现，每次父组件因为某个 input 的 `value` 变化而重渲时，所有子组件依然会跟着重渲。
- **原因**：因为每次父组件渲染时，都会重新创建一个新的函数实例，导致传递给子组件的 props（函数引用）发生了变化。
- **解决方案**：我们使用 `useCallback` 来包裹这些事件处理函数。`useCallback` 会缓存函数实例，只有当它的依赖项（通常是函数内部用到的 state 或 props）变化时，才会重新创建函数。这样，传递给子组件的 prop 引用就保持了稳定，避免了无效渲染。同理，对于复杂的计算，我们会使用 `useMemo` 来缓存计算结果。

3. 列表性能优化：虚拟列表 (Virtualization):

- **实践**：我们有一个需要展示上千条日志记录的页面。如果一次性将所有日志都渲染成 DOM 节点，会导致页面创建时白屏时间很长，并且滚动时会非常卡顿。
- **解决方案**：我们引入了 `react-window` 这个库来实现**虚拟列表**。它的原理是，**只渲染视口 (Viewport) 内可见区域的列表项**。当用户滚动时，它会动态地计算和替换视口内的列表项，而视口外的 DOM 节点则被回收。这样，无论列表总数据量有多大，实际渲染的 DOM 节点数量始终是有限的，从而彻底解决了长列表的性能问题。

4. 代码分割与懒加载 (Code Splitting & Lazy Loading):

- **实践**：我们的单页应用（SPA）非常庞大，包含了管理后台、用户中心、数据报表等多个模块。如果将所有代码都打包到一个 JS 文件里，首屏加载会非常慢。

- **解决方案：**我们使用 `React.lazy` 和 `Suspense` 对路由组件进行了代码分割。当用户访问某个路由时，对应的组件代码才会被动态加载。同时，我们使用 `Suspense` 并传入一个 `fallback` UI（比如一个加载中的 loading spinner），来提供一个友好的加载过渡体验。

二、构建层面（Webpack/Vite 优化）

- **Tree Shaking：**确保在生产模式下开启，移除未被引用的代码。
- **代码压缩：**使用 `TerserWebpackPlugin` 压缩 JS，`CssMinimizerWebpackPlugin` 压缩 CSS。
- **分析打包体积：**定期使用 `webpack-bundle-analyzer` 分析打包结果，找出体积过大的第三方库（比如 `Moment.js`，我们后来换成了更轻量的 `Day.js`），或者检查是否有模块被重复打包。

三、监控层面

- **使用 Profiler API：**React DevTools 提供了一个强大的 Profiler 工具。在开发阶段，我会用它来录制应用的交互过程，然后分析哪些组件的渲染开销最大、哪些组件发生了不必要的渲染，从而有针对性地进行优化。
- **性能指标监控：**在生产环境，我们会接入前端性能监控平台，关注一些关键指标，如 LCP (Largest Contentful Paint)、FID (First Input Delay) 等，以便及时发现和定位线上的性能问题。

通过这样一套组合拳，我们可以系统性地提升 React 应用的性能和用户体验。

如果一个列表页，有大量数据需要渲染，如何防止页面卡顿？

这个问题刚才在性能优化部分其实已经提到了，我可以再深入和系统地展开一下。处理大数据量列表渲染的核心思路是：**避免一次性将所有数据都渲染成真实的 DOM 节点。**

具体有以下几种方案，可以根据场景复杂度递进使用：

1. 分页 (Pagination)：

- **描述：**这是最传统也是最简单的方案。将大量数据分割成多个页面，每页只加载和渲染固定数量的数据（比如 20 条）。
- **优点：**实现简单，逻辑清晰，对前端和后端的压力都比较小。
- **缺点：**用户需要频繁点击“下一页”来浏览数据，体验上可能不够流畅。
- **适用场景：**管理后台的数据表格、搜索结果列表等。

2. 无限滚动 (Infinite Scrolling)：

- **描述：**当用户滚动到列表底部时，自动加载下一页的数据并追加到当前列表的末尾。
- **优点：**提供了“无缝”的浏览体验，用户无需手动翻页。
- **缺点：**随着滚动加载的数据越来越多，DOM 节点数量会持续增加，最终还是会导致页面滚动性能下降和内存占用过高。
- **适用场景：**信息流、社交媒体动态墙（如微博、Twitter）。

3. 虚拟列表/窗口化 (Virtualization / Windowing):

- **描述:** 这是解决长列表性能问题的**终极方案**。它只渲染用户当前视口（可见区域）内的数据项，以及视口上下方少量用于缓冲的“预备”项。
- **工作原理:**
 - i. 容器元素的高度被设置为所有列表项的总高度（`总数 * 单项高度`），这样浏览器的滚动条就能正常工作。
 - ii. 内部有一个绝对定位的列表元素，通过 `transform: translateY()` 属性来模拟滚动效果。
 - iii. 监听容器的 `scroll` 事件，根据 `scrollTop` 值，动态计算出当前视口内应该显示哪些数据项。
 - iv. 只将这些计算出的数据项渲染成真实的 DOM 节点。
- **优点:** 无论总数据量是 1000 条还是 10 万条，真实渲染的 DOM 节点数量始终是固定的（比如 20 个）。这使得页面的初始渲染速度极快，滚动也极为流畅。
- **缺点:** 实现起来相对复杂，尤其是在列表项高度不固定的情况下，需要动态计算和缓存每项的高度和位置。
- **实现:** 通常不建议自己从零实现，可以直接使用成熟的第三方库，如 `react-window` (用于固定高度列表) 或 `react-virtualized` (功能更强大，支持动态高度)。

总结方案选择:

- 对于**数据需要被精确索引和分页**的场景，选择**分页**。
- 对于**追求流畅浏览体验，且总数据量不是特别极端**的场景（比如几百条），可以选择**无限滚动**。
- 对于**总数据量巨大（上千甚至上万条）**，并且对性能要求极高的场景，**虚拟列表**是唯一的正确选择。

在实践中，我们甚至可以将**无限滚动**和**虚拟列表**结合起来：当用户滚动到底部时，触发 API 请求加载更多数据，并将新数据追加到数据源中，而虚拟列表的渲染机制确保了 UI 的流畅性。

前端工程化与团队协作

你对前端工程化是怎么理解的？你在团队里是如何实践的？

我对前端工程化的理解：

我认为，前端工程化不是指某一项具体的技术，而是一套**思想和体系**。它的核心目标是**提升前端团队的开发效率、保障项目的代码质量、降低项目的维护成本**。它借鉴了软件工程的理念，将前端开发过程中的各个环节——从创建、开发、编译、测试、部署到运维——都通过一系列的工具和流程进行规范化和自动化。

前端工程化主要关注以下几个方面：

1. 规范化：

- **代码规范**：统一的编码风格（ESLint）、CSS 命名（BEM）、Git Commit Message 规范等。
- **项目结构**：统一的目录结构、组件设计规范。
- **流程规范**：统一的分支管理模型（Git Flow）、Code Review 流程、发布流程。

2. 模块化与组件化：

- **模块化**：将复杂的系统拆分成独立的、可复用的模块（JS模块、CSS模块）。
- **组件化**：将 UI 拆分成独立的、可复用的组件，提高开发效率和可维护性。

3. 自动化：

- **自动构建**：自动化地进行代码编译、压缩、打包（Webpack/Vite）。
- **自动测试**：自动化地运行单元测试、集成测试。
- **自动部署**：通过 CI/CD (Continuous Integration/Continuous Deployment) 流程，实现代码提交后自动构建、测试和部署。

我们在团队里的实践：

我们团队围绕“提效、保质”这两个核心点，建立了一套完整的前端工程化体系。

1. 项目创建阶段：脚手架 (CLI)

- 我们基于 Plop.js 定制了一个内部的 CLI 工具。开发者可以通过 `npm run new:page` 或 `npm run new:component` 这样的命令，一键生成符合我们团队规范的页面或组件模板。模板中已经预设好了目录结构、初始代码、样式文件、测试文件等，避免了重复的“复制粘贴”工作，也保证了项目结构的一致性。

2. 开发阶段：规范与工具

- **代码规范**：我们使用 **ESLint + Prettier + Stylelint**。ESLint 负责保证代码质量和逻辑错误，Prettier 负责统一代码格式，Stylelint 负责 CSS 规范。
- **Git 流程**：我们集成了 **Husky + lint-staged + commitlint**。
 - `pre-commit` 钩子：在 `git commit` 之前，`lint-staged` 会自动对暂存区的文件运行 ESLint 和 Prettier 检查并自动修复，确保提交到仓库的代码都是符合规范的。
 - `commit-msg` 钩子：`commitlint` 会检查 commit message 是否符合我们约定的格式（如 `feat: add login page`），不符合则无法提交。这保证了 Git 历史的可读性。

3. 测试阶段：自动化测试

- 我们要求核心的工具函数和复杂的业务组件必须编写**单元测试**。我们使用 **Jest** 作为测试框架，使用 **React Testing Library** 来测试组件。
- 这些测试用例被集成到 CI/CD 流程中，每次提交代码或发起 Merge Request 时，都会自动运行所有测试。如果测试不通过，合并请求就会被阻止，从而保证了代码逻辑的正确性。

4. 构建与部署阶段：CI/CD

- 我们使用 **Jenkins**（或 GitLab CI）来搭建 CI/CD 流水线。
- **开发分支（develop）**：当代码合并到 `develop` 分支时，会自动触发 CI 流程，执行代码检查、单元测试、构建，并将产物部署到测试环境。
- **主分支（main）**：当代码合并到 `main` 分支时（通常通过发布分支），会触发生产环境的构建和部署流程，自动将应用发布到线上。

通过这套体系，我们将大量重复、易错的工作自动化，让开发者可以更专注于业务逻辑的实现，同时也极大地保障了整个项目的稳定性和可维护性。

你们的代码质量是如何保障的？比如 Code Review, ESLint, Unit Test 等。

我们通过一个多层次、相结合的体系来保障代码质量，主要包括**事前预防**、**事中发现**和**事后保障**三个环节。

5. 事前预防：规范与工具

- **ESLint & Prettier & Stylelint**：这是我们的第一道防线。我们制定了严格的 ESLint 规则集（基于 Airbnb 规范并进行了一些定制），并结合 Prettier 强制统一代码风格。这些工具被集成到 VSCode 编辑器插件和 `pre-commit` 钩子中。
 - **编辑器插件**：在开发者编写代码时，就能实时地发现并提示不规范或有潜在错误的代码，并可以自动格式化。
 - **Git Hooks (Husky + lint-staged)**：在代码提交前，强制对本次提交的代码进行检查和格式化。如果不符合规范，提交将被拒绝。这从源头上保证了进入代码库的代码至少在风格和基础质量上是合格的。

6. 事中发现：Code Review

- **Code Review (CR)** 是我们保障代码质量最核心的环节。我们遵循以下流程：
 - **发起时机**：所有的功能开发、bug 修复，在合并到主开发分支 (`develop`) 之前，都必须通过 Merge Request (MR) 的形式发起 Code Review。
 - **Reviewer 指派**：至少需要一位以上的同事进行 Review，通常是项目的核心成员或者对相关业务更熟悉的同事。
 - **Review 内容**：CR 不仅仅是检查代码风格（这部分工作已经交给 ESLint 了），我们更关注：
 - **逻辑正确性**：代码是否正确地实现了需求？有没有考虑边界情况？

- **代码可读性与可维护性**：变量/函数命名是否清晰？代码是否有必要的注释？逻辑是否过于复杂，能否被简化？
- **性能与安全**：是否存在潜在的性能瓶颈（如循环中的昂贵操作）？是否有安全隐患（如 XSS）？
- **架构与设计**：代码是否遵循了项目的设计模式？组件的拆分是否合理？
- **工具支持**：我们使用 GitLab 的 Merge Request 功能进行在线 CR，可以方便地对具体代码行进行评论和讨论。MR 只有在所有讨论都解决（resolved）并且获得至少一个 `Approve` 之后，才能被合并。

7. 事后保障：自动化测试

- **单元测试 (Unit Test)**：我们使用 Jest 和 React Testing Library。
 - **覆盖范围**：我们强制要求所有公共的工具函数、Hooks 以及核心的、复用性高的业务组件必须编写单元测试。
 - **目的**：保证单个模块的功能是正确的，并且在未来的代码重构或修改中，能够通过运行测试快速验证是否引入了新的 bug（回归测试）。
- **端到端测试 (E2E Test)**：对于一些核心的用户流程，比如登录、注册、下单，我们还会编写 E2E 测试（使用 Cypress）。这些测试会模拟真实用户在浏览器中的操作，来保证整个应用的流程是通畅的。
- **CI 集成**：所有的测试都被集成在我们的 CI/CD 流水线中。每次提交代码都会自动触发测试运行，如果任何一个测试用例失败，构建过程就会中断，代码也无法被合并或部署。

总结：我们的代码质量保障体系是一个金字塔结构。**ESLint** 等工具是基石，保证了最基础的代码规范；**Code Review** 是核心，通过人和人之间的交流来保证代码的逻辑和设计质量；**自动化测试**是顶层的安全网，为项目的长期稳定性和回归提供了最终保障。

讲一下微前端，了解吗？它的实现方案有哪些？

了解。**微前端 (Micro Frontends)** 是一种架构风格，它借鉴了微服务的思想，**将一个庞大、单一的前端应用 (Monolithic Front-end) 拆分成多个更小、更独立、可以自治的前端应用**。这些小的应用可以被独立地开发、测试、部署，然后再被组合起来，共同构成一个完整的用户体验。

它主要解决了什么问题？

随着前端应用变得越来越复杂，单一应用会面临诸多挑战：

- **技术栈陈旧，难以升级**：整个应用被锁定在一个技术栈上（比如旧版的 Angular 或 React），想要升级或引入新技术，成本极高。
- **开发效率低下**：所有开发者都在一个巨大的代码库上工作，代码耦合度高，互相影响，构建和部署速度慢。

- **团队协作困难**：不同团队之间的协作变得复杂，一个小的改动可能需要协调多个团队。
- **部署风险高**：任何一个小的改动都需要重新部署整个应用，增加了风险。

微前端的核心价值就是通过拆分，实现**技术栈无关、独立开发、独立部署**，从而解决上述问题。

主流的实现方案有哪些？

实现微前端的关键在于如何将这些独立的应用**组合（或编排）**起来，主要有以下几种方案：

1. iFrame 方案：

- **描述**：这是最简单、最古老的方案。主应用通过 `<iframe>` 标签来嵌入各个子应用。
- **优点**：天然的沙箱隔离机制，CSS 和 JavaScript 互相不影响，实现简单。
- **缺点**：
 - **体验差**：`iframe` 会阻塞主页面的 `onload` 事件；URL 不同步，浏览器前进后退按钮体验不好；UI 适配困难，弹窗、loading 等都会被限制在 `iframe` 内部。
 - **通信复杂**：父子应用通信需要通过 `postMessage`，比较繁琐。
 - **性能问题**：每个 `iframe` 都是一个独立的文档，会创建独立的浏览器上下文，内存开销大。

2. Web Components 方案：

- **描述**：利用 Web Components 的标准（如 Custom Elements, Shadow DOM），将每个子应用封装成一个自定义 HTML 标签。主应用像使用普通 HTML 标签一样使用它们。
- **优点**：技术标准，无框架依赖；自带样式和脚本隔离（Shadow DOM）。
- **缺点**：存在一定的浏览器兼容性问题；需要对 Web Components 技术有一定了解。

3. Single-SPA (JavaScript Entry) 方案：

- **描述**：这是一个非常流行的“路由分发式”微前端框架。它提供了一个主框架，负责注册所有的子应用。当路由变化时，主框架会根据路由规则，去加载并挂载对应的子应用。
- **工作流程**：主应用定义好路由和应用的对应关系。当 URL 变化时，Single-SPA 会判断哪个子应用应该被激活（`active`），然后去加载这个应用的 JS 文件，并调用其暴露的生命周期函数（`bootstrap`，`mount`，`unmount`）来启动或卸载应用。
- **优点**：技术栈无关，支持 React, Vue, Angular 等任意框架；实现了解耦。
- **缺点**：需要子应用进行一定的改造来配合主框架的生命周期；CSS 隔离和 JS 沙箱需要自己额外处理。

4. qiankun (基于 Single-SPA) 方案：

- **描述**：`qiankun` 是由蚂蚁集团开源的，基于 Single-SPA 的微前端实现库。它在 Single-SPA 的基础上，提供了更多开箱即用的能力，被认为是目前最成熟的方案之一。
- **优点**：

- **HTML Entry**: 可以直接加载子应用的 HTML 文件作为入口，接入成本极低，子应用几乎无需改造。
- **开箱即用的沙箱机制**: 提供了 JS 沙箱（Proxy）和样式隔离（Shadow DOM 或动态样式表），有效解决了子应用间的全局变量污染和样式冲突问题。
- **资源预加载**: 在浏览器空闲时，可以预加载其他未激活的子应用资源，提升切换速度。
- **缺点**: 目前在国内社区非常流行，但在国际社区的知名度相对较低。

5. Module Federation (模块联邦) 方案:

- **描述**: 这是 Webpack 5 推出的一个革命性功能。它允许一个 JavaScript 应用在运行时动态地加载另一个独立应用的代码。
- **工作原理**: 每个应用既可以作为“宿主 (Host)”，消费其他应用的模块；也可以作为“远程 (Remote)”，暴露自己的模块给别人使用。这种依赖关系是在运行时建立的，而不是编译时。
- **优点**: 真正的去中心化，没有“主应用”和“子应用”的概念，每个应用都可以是平等的；可以实现应用间的组件、函数共享，非常灵活；依赖共享做得很好，可以避免公共库被重复加载。
- **缺点**: 强依赖 Webpack 5，有技术栈限制；配置相对复杂。

总结: 目前，qiankun 和 Module Federation 是社区中最流行和最受推荐的两种微前端实现方案。qiankun 更像一个完整的解决方案，而 Module Federation 则提供了更底层、更灵活的能力。

Web 基础与安全

SSR（服务端渲染）和 CSR（客户端渲染）有什么区别？你在项目中用过吗？

SSR 和 CSR 是两种主流的 Web 应用渲染模式，它们在渲染时机、性能表现和 SEO 上有本质的区别。

特性	CSR (Client-Side Rendering)	SSR (Server-Side Rendering)
渲染地点	浏览器 (客户端)	服务器
	1. 浏览器请求	1. 浏览器请求 HTML。 2. 服务器接收请求，在服务端执行 JS (React)

工作流程	HTML。 2. 服务器返回一个 空的 HTML 骨架 和一个 JS bundle 文件。 3. 浏览器下载并执行 JS。 4. JS 代码（如 React）在浏览器中执行，生成 DOM，并挂载到页面上。	代码 ，将组件渲染成完整的 HTML 字符串。 3. 服务器将这个 包含内容的 HTML 返回给浏览器。 4. 浏览器直接展示这个 HTML。 5. 同时下载 JS 文件，JS 执行后会进行 "Hydration" (注水)，即接管页面上的 DOM，使其能够响应用户交互。
首屏加载速度 (FCP)	慢 。用户需要等待 JS 文件下载并执行完后才能看到页面内容，在此之前通常是白屏或 loading 动画。	快 。浏览器收到的是完整的 HTML，可以立即解析和渲染，用户能更快地看到页面内容。
可交互时间 (TTI)	FCP 和 TTI 几乎是同时的。一旦 JS 执行完，页面既可见又可交互。	可能较慢 。虽然 FCP 很快，但用户看到页面后，页面可能还不能立即响应交互，需要等待 JS 下载并执行完 "Hydration" 过程。
SEO 友好度	差 。搜索引擎爬虫可能无法正确执行 JS，导致抓取到的是空内容，不利于 SEO。虽然现在 Google 爬虫已经能很好地执行 JS，但并非所有爬虫都可以。	好 。返回给爬虫的是完整的 HTML 内容，非常有利于搜索引擎优化。
服务器压力	小 。服务器只负责提供静态文件，大部分计算和渲染工作都由客户端承担。	大 。每次请求，服务器都需要进行一次渲染，对 CPU 和内存的消耗都比较大。
技术选型	典型的 SPA (Single Page Application) 应用，如使用 Create React App 创建的项目。	Next.js, Nuxt.js 等框架。

我在项目中用过吗？

是的，我在一个**内容型**的网站（类似博客或资讯门户）中使用了 SSR。

- **项目背景**：这个项目对 **SEO** 和 **首屏加载速度** 有非常高的要求。我们需要让搜索引擎能够轻松地抓取到每一篇文章的内容，同时也希望用户能在访问 URL 后第一时间看到文章，而不是一个 loading 动画。
- **技术选型**：我们选择了 **Next.js** 这个基于 React 的 SSR 框架。
- **实践效果**：
 - a. **SEO 效果显著**：通过 SSR，我们网站的页面能够被搜索引擎完美收录，带来了很好的自然流量。
 - b. **首屏性能提升**：用户访问页面的 FCP 时间大大缩短，体验得到了明显改善。
 - c. **开发体验**：Next.js 提供了非常好的开发体验，它内置了文件系统路由、数据获取 API (`getServerSideProps`) 等功能，让我们能够很方便地实现服务端渲染，而无需自己配置复杂的 Webpack 和 Node.js 服务器。

总结：CSR 更适合内部系统、管理后台等对 SEO 要求不高、交互逻辑复杂的应用。而 SSR 则非常适合官网、博客、电商网站等对 SEO 和首屏性能有要求的“内容驱动型”应用。当然，现在还有 SSG (Static Site Generation) 和 ISR (Incremental Static Regeneration) 等更先进的渲染模式，它们在 SSR 的基础上做了更多的性能优化。

Web 安全问题，除了 XSS 和 CSRF，还了解其他的攻击方式吗？比如点击劫持。

是的，除了常见的 XSS (跨站脚本攻击) 和 CSRF (跨站请求伪造)，Web 安全领域还有很多其他的攻击方式。

点击劫持 (Clickjacking)

- **攻击原理**：攻击者创建一个自己的恶意网站，然后使用一个**透明的** `<iframe>` 将目标网站（比如一个银行转账页面）覆盖在恶意网站之上。接着，攻击者会用一些诱导性的内容（比如“点击这里领取奖品”）来欺骗用户点击。用户以为自己点击的是“领取奖品”按钮，但实际上他们点击的是隐藏在下方的 `<iframe>` 中的“确认转账”按钮。
- **防御方式**：
 - **HTTP 响应头 `X-Frame-Options`**：这是最主要的防御手段。它可以告诉浏览器一个页面是否允许在 `<frame>`，`<iframe>`，`<embed>` 或 `<object>` 中展现。
 - **DENY**：页面不允许在任何 frame 中展示。
 - **SAMEORIGIN**：页面只允许在同源的 frame 中展示。

- `ALLOW-FROM uri` : 页面只允许在指定的 uri 的 frame 中展示（这个指令有兼容性问题，不推荐）。
- **Frame Busting (JS防御)**: 在页面中加入一段 JavaScript 代码，判断 `top.location !== self.location`，如果成立，说明页面被嵌入到了 `iframe` 中，则可以强制让顶层窗口跳转到自己的 URL。但这种方法可以被绕过，不如 `X-Frame-Options` 可靠。

其他常见的攻击方式：

- **SQL 注入 (SQL Injection)**:
 - **原理**: 攻击者在应用的输入框中，提交一段恶意的 SQL 查询代码。如果后端没有对输入进行严格的过滤和转义，直接将输入拼接到了 SQL 语句中，那么这段恶意的 SQL 代码就会被数据库执行，从而导致数据泄露、篡改或删除。
 - **防御**: 使用**参数化查询**（Prepared Statements），永远不要手动拼接 SQL 语句。对用户输入进行严格的验证和清理。
- **中间人攻击 (Man-in-the-Middle, MITM)**:
 - **原理**: 攻击者将自己置于用户和服务器之间，拦截并可能篡改双方的通信内容。这通常发生在不安全的网络环境（如公共 Wi-Fi）中，特别是当通信使用明文的 HTTP 协议时。
 - **防御**: 全站启用 **HTTPS**。HTTPS 通过 SSL/TLS 协议对通信内容进行加密，并验证服务器的身份，可以有效防止中间人攻击。
- **拒绝服务攻击 (Denial-of-Service, DoS / DDoS)**:
 - **原理**: 攻击者通过发送大量无效或高负载的请求，耗尽服务器的带宽、CPU 或内存资源，使得服务器无法响应正常用户的请求。DDoS (Distributed DoS) 是指攻击者控制大量的“僵尸网络”从不同地方同时发起攻击。
 - **防御**: 这是一个复杂的系统工程，通常需要依赖专业的网络安全设备和云服务提供商（如 CDN、WAF 防火墙）来清洗流量、限制请求频率、识别和封禁恶意 IP。
- **文件上传漏洞**:
 - **原理**: 如果服务器允许用户上传文件，但没有对文件的类型、大小、内容做严格的校验，攻击者就可能上传一个可执行的脚本文件（如 PHP、JSP webshell），然后通过访问这个文件，获得服务器的控制权。
 - **防御**: 严格限制上传文件的类型（白名单策略），对文件名进行重命名，将文件存储在与 Web 服务器分离的文件服务器上，并设置不可执行权限。

HTTPS 的加密过程是怎样的？为什么需要数字证书？

HTTPS (Hyper Text Transfer Protocol Secure) 的核心是在 HTTP 的基础上，加入了一层 SSL/TLS 安全协议。它的加密过程结合了**对称加密**和**非对称加密**两种方式，以兼顾安全性和性能。

为什么需要数字证书？

在解释加密过程之前，必须先理解数字证书的作用。假设客户端要和服务器通信，如何**确保正在通信的服务器就是它声称的那个服务器**，而不是一个伪装的中间人？

数字证书 (Digital Certificate) 就是解决这个问题的。它由一个权威的、受信任的**第三方机构——证书颁发机构 (Certificate Authority, CA)** 签发，相当于服务器的“网络身份证”。

证书中包含了以下关键信息：

- **公钥 (Public Key)**：服务器的公钥。
- **持有者信息**：网站的域名、所有者等。
- **CA 的数字签名**：CA 使用自己的私钥对证书信息进行加密生成一个签名。

当浏览器收到证书时，会使用操作系统或浏览器中**内置的 CA 的公钥**来解密这个签名。如果解密成功，并且解密后的信息与证书信息一致，浏览器就能确认：1) 这个证书确实是由这个**可信**的 CA 颁发的；2) 证书内容没有被篡改。从而验证了服务器身份的真实性。

HTTPS 的加密过程 (TLS 握手过程简化版)：

1. 客户端 Hello (Client Hello)：

- 客户端（浏览器）向服务器发起请求，并发送一个 "Hello" 消息。
- 消息中包含：客户端支持的 TLS 版本、一个**随机数 (Client Random)**、客户端支持的加密算法套件列表。

2. 服务器 Hello (Server Hello) 与证书：

- 服务器收到请求后，回复一个 "Hello" 消息。
- 消息中包含：确认使用的 TLS 版本和加密算法、另一个**随机数 (Server Random)**。
- **最重要的一步**：服务器会将自己的**数字证书**发送给客户端。

3. 客户端验证与密钥交换：

- 客户端收到服务器的证书后，会进行验证（如上文所述）。
- 验证通过后，客户端会再生成一个**随机数 (Pre-Master Secret)**。
- 客户端使用从证书中获取的**服务器公钥**，对这个 **Pre-Master Secret** 进行**加密**，然后发送给服务器。

4. 服务器解密与生成会话密钥：

- 服务器收到被加密的 **Pre-Master Secret** 后，使用自己的**私钥**进行解密，得到原始的 **Pre-Master Secret**。
- 现在，**客户端和服务器双方都拥有了三个相同的随机数**：Client Random, Server Random, Pre-Master Secret。
- 双方会使用一个**相同的约定好的算法**，通过这三个随机数，各自独立地计算出本次通信所使用的**“会话密钥 (Session Key)”**。

5. 握手完成，开始加密通信：

- 客户端和服务端各自向对方发送一个“握手完成”的消息，这个消息本身就是用刚刚生成的**会话密钥**进行加密的。
- 如果双方都能成功解密对方的消息，则 TLS 握手过程完成。
- 之后的所有 HTTP 数据，都将使用这个**会话密钥**进行**对称加密**来进行传输。

总结：

- **非对称加密**（公钥/私钥）被用在**握手阶段**，主要作用是**安全地协商**出后续通信用的对称密钥，并**验证服务器身份**。它的计算开销大，不适合加密大量数据。
- **对称加密**（会话密钥）被用在**数据传输阶段**，作用是**加密**实际的 HTTP 请求和响应内容。它的计算速度快，性能好。
- **数字证书**的作用是在非对称加密开始之前，**保证客户端获取到的公钥是真实、可信的**，从而防止中间人攻击。

Node.js 在前端开发中扮演什么角色？你用它做过什么？

Node.js 对现代前端开发来说是不可或缺的基石，它已经深深地融入到了前端工程化的每一个环节。它主要扮演以下几个角色：

1. 前端工程化的运行环境：

- 几乎所有的前端构建工具，如 **Webpack, Vite, ESLint, Babel, Sass/Less 编译器** 等，都是基于 Node.js 开发的。我们需要 Node.js 环境来运行这些工具，从而实现代码的编译、打包、检查和优化。可以说，**没有 Node.js，就没有现代前端工程化**。

2. 包管理器的基础：

- **npm** (Node Package Manager) 和 **yarn, pnpm** 等包管理器，让我们能够方便地安装、管理和共享项目依赖的第三方库。这些包管理器本身就是基于 Node.js 的。

3. 本地开发服务器：

- 像 `webpack-dev-server` 或 Vite 的开发服务器，都是用 Node.js 编写的。它们可以在本地启动一个 HTTP 服务器，提供热更新 (HMR)、API 代理等功能，极大地提升了我们的开发效率。

4. 服务端渲染 (SSR) 的后端宿主：

- 对于像 Next.js, Nuxt.js 这样的 SSR 框架，它们需要在服务器端执行 JavaScript/React/Vue 代码来生成 HTML。这个服务器端的执行环境，就是 Node.js。

5. BFF (Backend for Frontend, 服务于前端的后端)：

- 在复杂的项目中，后端的微服务可能提供了非常原子化、零散的 API 接口。前端为了渲染一个页面，可能需要调用多个微服务接口。这时，可以由前端团队自己维护一个 Node.js 写的 BFF 层。
- BFF 的作用是：**聚合和裁剪**后端的微服务 API，将多个请求合并成一个，并处理数据，最终向上层的前端应用（Web, iOS, Android）提供一个更友好、更符合页面展示需求的 API 接口。这可以减少前端的请求次数，并将一部分 UI 相关的逻辑从前端移到 BFF 层。

我用 Node.js 做过什么？

除了日常使用它作为工程化的基础环境外，我还用它做过以下一些具体的事情：

- **搭建 BFF 层**：在我们之前的一个电商项目中，商品详情页需要同时调用商品服务、库存服务、评论服务、推荐服务等多个微服务。我们使用 **Koa.js** (一个轻量的 Node.js 框架) 搭建了一个 BFF 层，它会接收前端的单个请求，然后在服务端并行地调用这几个微服务，将数据聚合处理后，一次性返回给前端。这大大简化了前端的逻辑，并提升了页面的加载性能。
- **编写脚手架 (CLI) 工具**：为了统一团队的项目规范，我使用 **Commander.js** 和 **Inquirer.js** 这两个库，编写了一个内部的 CLI 工具。它可以帮助团队成员快速创建符合我们规范的项目、页面或组件，实现了项目初始化的自动化。
- **简单的 Mock Server**：在和后端并行开发时，后端接口还没有准备好。我会使用 **Express.js** 快速搭建一个 Mock Server，根据约定好的 API 文档，返回模拟的 JSON 数据，这样前端就可以独立地进行开发和调试，而不会被阻塞。

场景题与算法题

场景题：现在要实现一个大型电商网站的商品详情页，你会如何设计这个页面？需要考虑哪些技术点，比如性能、可用性、SEO 等。

设计一个大型电商网站的商品详情页（Product Detail Page, PDP）是一个非常综合性的任务，需要平衡用户体验、技术性能和商业目标。我会从以下几个维度来考虑：

一、核心技术选型与架构

1. 渲染模式：SSR (或 SSG/ISR) 优先

- **原因**：商品详情页对 **SEO** 和 **首屏性能** 的要求是最高的。我们需要搜索引擎能抓取到完整的商品信息，也需要用户能立刻看到页面内容。
- **方案**：使用 **Next.js** 框架。

- 对于热门、访问量大的商品，可以使用 **SSG (静态站点生成)** 在构建时就生成好静态 HTML 页面，提供极致的加载速度。
- 对于海量的、不那么热门的商品，可以使用 **ISR (增量静态再生)**，在用户第一次访问时生成页面并缓存，后续用户直接访问缓存，同时可以设置一个过期时间，定期重新生成页面以保证数据新鲜度。
- 对于需要实时数据的部分（如库存、价格），则可以在客户端进行异步请求。

2. 组件化设计

- 将整个页面拆分成多个高内聚、低耦合的组件，如：
 - `<ProductGallery>`：商品图片画廊，支持缩放、切换。
 - `<ProductInfo>`：商品标题、价格、SKU 选择。
 - `<InventoryStatus>`：库存状态及所在地。
 - `<AddToCart>`：加入购物车按钮及数量选择。
 - `<ProductDescription>`：商品图文详情。
 - `<CustomerReviews>`：用户评价及评分。
 - `<Recommendations>`：相关商品推荐。
- 这种拆分有利于团队协作、代码复用和维护。

二、性能优化 (Performance)

1. 图片优化：图片是详情页最大的流量杀手。

- **图片懒加载 (Lazy Loading)**：首屏外的图片（尤其是商品详情长图）都应该懒加载。
- **响应式图片**：使用 `<picture>` 标签或 `srcset` 属性，根据不同的屏幕尺寸和分辨率加载不同大小的图片。
- **现代图片格式**：使用 WebP 或 AVIF 格式，可以在保证质量的同时，大大减小图片体积。
- **CDN 加速**：所有静态资源（图片、JS、CSS）都应该上 CDN。

2. 代码优化：

- **代码分割**：按组件进行代码分割，比如“用户评价”、“商品推荐”这些在页面下方的组件，可以等用户滚动到附近时再懒加载对应的 JS 代码。
- **关键 CSS (Critical CSS)**：将渲染首屏内容所必需的 CSS 内联到 HTML 的 `<head>` 中，以最快速度完成首次绘制。非关键的 CSS 则可以异步加载。

3. 数据请求优化：

- **BFF 层**：如前所述，使用 BFF 聚合后端多个微服务的 API，减少前端的 HTTP 请求数。
- **缓存**：对不经常变化的商品信息（如描述、参数）进行有效缓存（CDN 缓存、浏览器缓存）。对于实时性要求高的数据（库存、价格），则通过客户端脚本发起异步请求。

三、可用性与用户体验 (Usability & UX)

1. 交互反馈：

- **骨架屏 (Skeleton Screens)**：在等待数据加载时，显示页面的大致布局轮廓，而不是一个空白页或 loading 图标，体验更好。
- **操作反馈**：点击“加入购物车”后，应有明确的视觉反馈（如按钮 loading 状态、弹出成功提示、购物车图标变化）。
- **乐观更新 (Optimistic UI)**：点击“加入购物车”时，可以先假设操作成功并立即更新 UI，然后再向服务器发请求。如果请求失败，再回滚 UI 状态并提示用户。

2. 无障碍访问 (Accessibility, A11y)：

- 保证所有图片都有有意义的 `alt` 标签。
- 所有可交互元素（按钮、链接）都能通过键盘访问。
- 使用语义化的 HTML 标签。

3. 移动端优先 (Mobile First)：

- 设计和开发都应首先考虑移动端的用户体验，然后逐步扩展到桌面端。

四、SEO (搜索引擎优化)

- 结构化数据**：使用 [Schema.org](https://schema.org) 的标记（以 JSON-LD 格式嵌入）来描述商品信息，如商品名称、价格、评分、库存状况。这能帮助搜索引擎更好地理解页面内容，并可能在搜索结果中显示丰富的摘要 (Rich Snippets)。
- 语义化 HTML**：使用 `<h1>` 标签表示商品标题，`<h2>` 等表示章节标题。
- 友好的 URL**：URL 应该简洁、有意义，包含商品名称，如 `example.com/products/iphone-15-pro`。
- Meta 标签**：为每个商品页面生成唯一的、有吸引力的 `title` 和 `description` meta 标签。

五、可靠性与可用性 (Reliability & Availability)

- 降级处理**：如果某个非核心服务（如“商品推荐”服务）挂了，页面不应该崩溃，而应该能优雅地降级（比如直接隐藏推荐区域）。
- 错误处理**：对所有的 API 请求做完善的错误处理，并给用户清晰的提示。
- 监控与告警**：对页面的性能指标、JS 错误、API 成功率进行全面的监控，并设置告警。

通过综合考虑以上这些技术点，我们可以设计和实现一个高性能、高可用、用户体验良好且对搜索引擎友好的电商商品详情页。

算法题：给定一个无序的整数数组，找到其中最长连续序列的长度。

题目要求：

给定一个未排序的整数数组 `nums`，找出数字连续的最长序列（不要求序列元素在原数组中连续）的长度。要求算法的时间复杂度为 $O(n)$ 。

示例：

输入：nums = [100, 4, 200, 1, 3, 2]

输出：4

解释：最长数字连续序列是 [1, 2, 3, 4]。它的长度为 4。

解题思路：

如果我们先对数组排序，那么问题就变得很简单了，只需要遍历一次排序后的数组即可。但排序的时间复杂度至少是 $O(n \log n)$ ，不符合题目要求的 $O(n)$ 。

为了达到 $O(n)$ 的时间复杂度，我们可以使用一个以空间换时间的方法，借助哈希表（在 JavaScript 中就是 `Set` 对象）。

核心思想：

1. 首先，将数组中所有的数字存入一个 `Set` 中。这样做的好处是，我们可以在 $O(1)$ 的时间复杂度内判断一个数字是否存在。
2. 然后，遍历原始数组 `nums` 中的每一个数字 `num`。
3. 对于每个数字 `num`，我们检查 `Set` 中是否存在 `num - 1`。
 - 如果 `num - 1` **不存在**，说明 `num` 是一个**连续序列的起点**。
 - 如果 `num - 1` **存在**，说明 `num` 不是一个序列的起点（比如对于 3，因为 2 存在，所以我们不从 3 开始计算，而是会等到遍历到 1 时再开始计算整个序列），我们可以直接跳过，去检查下一个数字。
4. 当我们找到一个序列的起点 `num` 时，我们就从这个数字开始，不断地检查 `num + 1`，`num + 2`，... 是否存在于 `Set` 中，以此来计算当前连续序列的长度。
5. 在遍历过程中，我们维护一个变量 `maxLength` 来记录遇到的最长的连续序列长度。

为什么这个方法是 $O(n)$ 的？

虽然代码看起来有两层循环（外层遍历数组，内层 `while` 循环），但每个数字最多只会被访问两次：一次是在外层循环中被检查，一次是在 `while` 循环中作为序列的一部分被检查。因此，总的时间复杂度是线性的，即 $O(n)$ 。

代码实现：

代码块

```
1  /**
2   * @param {number[]} nums
3   * @return {number}
4   */
5  var longestConsecutive = function(nums) {
    // 如果数组为空，直接返回 0 if (nums.length === 0) {
```

```
6     return 0;
7 }
8
9 // 1. 将所有数字存入 Set，用于 O(1) 复杂度的查找const numSet = new Set(nums);
10
11 let maxLength = 0;
12
13 // 2. 遍历 Set 中的每个数字for (const num of numSet) {
14     // 3. 检查 num 是否是一个连续序列的起点// 如果 num - 1 存在，说明 num 不是起点，跳
    过if (!numSet.has(num - 1)) {
15         // 4. 如果 num 是起点，开始计算以此为起点的序列长度let currentNum = num;
16         let currentLength = 1;
17
18         // 不断检查下一个连续的数字是否存在while (numSet.has(currentNum + 1)) {
19             currentNum += 1;
20             currentLength += 1;
21         }
22
23         // 5. 更新最大长度
24         maxLength = Math.max(maxLength, currentLength);
25     }
26 }
27
28 return maxLength;
29 };
30
31 // --- 测试用例 ---const nums1 = [100, 4, 200, 1, 3, 2];
32 console.log(longestConsecutive(nums1)); // 输出: 4const nums2 = [0, 3, 7, 2, 5,
    8, 4, 6, 0, 1];
33 console.log(longestConsecutive(nums2)); // 输出: 9
```